Stacked Mixed-Order Graph Convolutional Networks for Collaborative Filtering

Hengrui Zhang* Julian McAuley†

Abstract

Graph-based recommendation algorithms treat user-item interactions as bipartite graphs, based on which low-dimensional vector representations of users and items seek to preserve the relationships among them. Previous methods usually capture users' preferences by directly learning first-order neighborhood patterns for each node, which limits their ability to exploit the similarity between two distant users/items as well as a user's preferences toward distant items. To address this potential weakness, in this paper, we propose *SMOG-CF* (Stacked Mixed-Order Graph Convolutional Networks for Collaborative Filtering), a GCN-based framework that can directly capture high-order connectivity among nodes. Instead of implicitly capturing high-order connectivity through embedding propagation, SMOG-CF facilitates 'path-level' information propagation between neighboring nodes at any order. The matrix form of our embedding propagation formulas yields a model that is easy to deploy and can be extended to a general framework by adopting various information construction and aggregation equations. Experiments on several datasets of varying scale demonstrate the efficacy of our model.

Keywords: Recommendation Systems, Network Embedding, Graph Convolutional Networks, High-order Connectivity.

1 Introduction

Recommender Systems play an important role in many services including advertising, e-commerce, and social media. Collaborative Filtering (CF) addresses recommendation problems by assuming that similar users typically share similar interests over items. Users and items can be embedded as low dimensional vectors according to historical user-item interaction data, such that the trained embeddings will be useful for predicting whether a user will interact with an item.

*Shanghai Jiao Tong Univerisity, China; University of California, San Diego, the United States. (sqstardust@sjtu.edu.cn)

†University of California, San Diego, the United States. (jmcauley@eng.ucsd.edu)

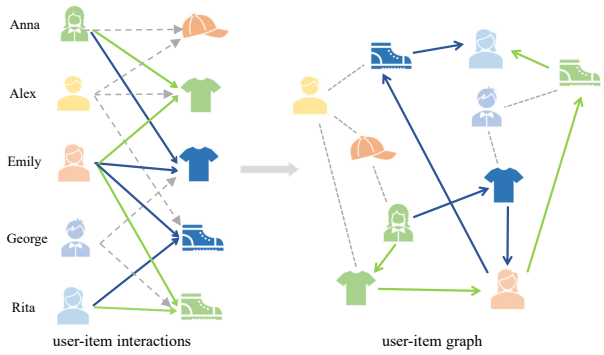

Figure 1: An example of high-order relationships in a user-item recommendation scenario. Left is the user-item interactions and right is the user-item graph. Blue and green lines are two 4-hop paths connecting Anna and Rita.

Following the development of Network Representation Learning, graph-based models have gained increasing attention. For recommendation, interactions can be modeled as a bipartite graph, where users and items are two isolated node sets and edges denote interactions between users and items. Graph-based models explore the local and global topological structure of the user-item graph combined with other attributes, and aim to learn efficient low-dimensional representations for each user and item. Graph Convolutional Networks (GCNs) [7] and their variants [4, 19–21] extend deep learning algorithms to graph-structured data by defining convolution operators on graphs, and have proven powerful when dealing with various downstream tasks [3, 13, 17, 22], including learning low-dimensional embeddings of users and items in a recommender system [19, 21, 26]. However, such models struggle to capture higher-order connectivity patterns among nodes, as they only aggregate information from directly neighboring nodes (or first-order neighbors), though it could be beneficial to take high-order connectivity into account [8, 15].

Fig. 1 illustrates the importance of high-order connections when inferring users' preferences toward items. Consider the following scenario, where customers are purchasing clothes. According to the user-item bipartite

Copyright © 2020 by SIAM
Unauthorized reproduction of this article is prohibited

graph presented, Anna and Emily share similar preferences, as they both bought blue and green shoes; similarly Rita shares preferences with Emily. Graph Convolutional Networks that exploit information from first-order neighbors can easily capture this type of information. However, consider the case where two users share no items in common. In Fig. 1, Anna and Rita have no common purchases but might still common share preferences. As highlighted in Fig. 1, we could find two paths between Anna and Rita: Anna $\rightarrow$ blue T-shirt $\rightarrow$ Emily $\rightarrow$ blue shoes $\rightarrow$ Rita (blue line), and Anna $\rightarrow$ green T-shirt $\rightarrow$ Emily $\rightarrow$ green shoes $\rightarrow$ Rita (green line). These two paths indicate that although Anna and Rita have no common items, there is a strong connection between these two users and they are likely to have similar preference to these items. This example suggests that multi-hop paths are vital for exploiting the high-order connectivity between two nodes in a graph and thus potentially helpful for inferring users' high-order preferences.

In this paper, we propose SMOG-CF, a framework for directly aggregating information from high-order neighbors of a target node in interaction graphs. We start with pair-wise message propagation from a p-order ($p > 1$) neighboring node to a target node, and demonstrate how to transform the pair-wise message propagation formula into a matrix form that can be deployed on large graphs, based on which SMOG-CF could be extended to other basic GCN models. Experiments on both large and small recommendation datasets with implicit feedback verify the effectiveness of our model.

Our contributions can be summarized as follows:

1. We propose SMOG-CF, a new CF model that seeks to learn the message propagation from a node's any-order neighbors in a user-item graph by defining a pair-wise message construction formula between two nodes that are not directly connected.

2. We present a derivation of the any-order embedding propagation formulas in matrix form, which makes our model straightforward and efficient to implement, based on which we could extend SMOG-CF to other GCN-based models.

3. We deploy SMOG-CF on four benchmark datasets. Our experiments verify the superiority of SMOG-CF over state-of-the-art techniques and the effectiveness of the proposed components.

2 Related Work

2.1 Network Representation Learning. Network representation learning techniques aim to learn low-dimensional latent representations of nodes in a network. Low-dimensional representations can effectively preserve local and global topological structures of a graph as well as node features, and can be used on downstream tasks, such as node classification and link prediction. Random-walk based models [2, 12, 18] were first proposed to learn such embeddings. Graph Neural Networks [4, 10, 20, 23], which try to adopt neural network methods on graph-structured data, have developed rapidly in recent years. Graph Convolutional Networks (GCNs) [7], which attempt to learn latent node representations by defining convolutional operation on graphs, were first proposed to solve semi-supervised classifications, and soon other GCN-based models were proposed for other tasks like social influence analysis [13], text classification [25] and recommender systems [19, 26]. Following previous studies [21, 24], in this paper we focus on learning user and item embeddings in a recommender (interaction) graph with implicit feedback.

2.2 High-order Proximity Learning on Graphs. Higher-order structure in graphs has proven beneficial in many cases such as hierarchical object representations, scene understanding, link prediction and recommender systems [1, 16, 21]. Network Motifs [15] were first proposed to learn such higher-order embeddings through Random-Walk based models, and were extended to Graph Neural Network structures by designing a convolutional layer with Motif attention that could aggregate first-order neighborhood information as well as high-order Motif information [8]. Based on GCNs, The high-order normalized Laplacian matrix is leveraged to aggregate information passed from any-order neighboring nodes [1, 9], though they fail to give a reasonable explanation as to why the high-order normalized Laplacian matrix should work in capturing connections across remote nodes. SMOG-CF directly captures the information passed from any of a node's p-order neighbors through a specific directed path, and demonstrates that the high-order normalized Laplacian matrix can be derived through message aggregation.

2.3 Graph-based Recommendation. As user-item interactions can be viewed as a bipartite graph, and a variety of studies have focused on using network embedding methods to learn user and item representations in recommender systems. [19] proposes a GCN-based auto-encoder framework for completing the user-item interaction matrix, though it only exploits first-order connections between nodes. PinSage [26] develops a data-efficient GCN structure which combines efficient random walks and graph convolutions to gener-

Copyright © 2020 by SIAM
Unauthorized reproduction of this article is prohibited

ate node embeddings. HOP-Rec [24] combines graph-based models with factorization models by performing random walks to enrich the interactions between high-order neighboring nodes and considering different orders of items simultaneously when decomposing the latent factors of user preferences. The recently proposed NGCF [21] stacks multiple Graph Convolution Layers to perform high-order embedding propagation and concatenates the output of each layer as the final representation of users and items, as an indirect way to leverage high-order connections, because this method cannot control the propagation strength between two nodes that are not directly connected. SMOG-CF is based on NGCF, though we devise a multi-hop information propagation path, based on which we can control the strength of message propagation between two high-order neighbors at any distance in an explicit way.

3 Preliminaries

3.1 Notations. In this paper, we focus on recommendation systems with implicit feedback, and we treat the user-item interaction as a bipartite graph. Following traditional recommendation models, we project each user or item to a low-dimensional embedding vector. Our task is to learn the best embedding vectors for each user and item and use these embeddings to make predictions. Table 1 introduces some of the most important symbols in this paper, while other notations will be introduced when mentioned.

Table 1: Notations

Symbol	Definition
$\mathcal{G}$	user-item bipartite graph
$\mathcal{U}$	user set
$\mathcal{V}$	item set
y_{ui}	interaction record of u and i
$\mathcal{N}_u$	user u's neighboring items
$\mathcal{N}_i$	item i's neighboring users
$\mathbf{A} = \{y_{ij}\}$	adjacency matrix of graph $\mathcal{G}$
$\mathbf{D} = \text{diag}(d_{ii})$	degree matrix of graph $\mathcal{G}$
$\mathbf{X} = \mathbf{E}^{(0)} = \{x_i\}$	initialized embedding matrix
$\mathbf{E}^{(l)} = \{e_i^{(l)}\}$	embedding matrix at layer l
$m_{i \leftarrow j}$	message propagated from j to i

3.2 General Graph Convolutional Networks. Here we introduce previous Graph Convolutional Network structures.

Graph Convolutional Networks try to define convolutional neural network structures on graph data, aiming to aggregate information from the neighborhoods of each node and propagate it to update the embedding of itself. This operation usually contains two parts: message construction and message aggregation. Message construction aims at calculating the information that each node receives from its neighboring nodes while message aggregation is aimed at delivering the integrated information as efficiently as possible so that the newly learned representation can better preserve topological information as well as the node's inherent attributes.

Graph Convolutional Network is an effective framework for representation learning on graphs, and many variants of GCNs have been proposed for different tasks. As introduced above, the representation of each node is updated using a pre-defined construction and aggregation operator to recursively aggregate the representation of its neighboring nodes. Formally, the l-th layer of a graph neural network can be generally defined as:

$$I_i^{(l)} = \text{AGGREGATE}\left(\left\{ m_{i \leftarrow j}^{(l)} : j \in \mathcal{N}_i \cup \{i\} \right\}\right), \tag{3.1}$$

where $m_{i \leftarrow j}^{(l)}$ is the message constructed that node j passed to node j at layer l, and can be formulated as:

$$m_{i \leftarrow j}^{(l)} = \text{CONSTRUCT}\left(e_j^{(l-1)}, e_i^{(l-1)}, c_{ij}\right), \tag{3.2}$$

where $e_i^{(l-1)}$ and $e_j^{(l-1)}$ represent the embeddings of node i and node j respectively. c_{ij} is a coefficient that reflects the connection strength between i and j. The choice of the function CONSTRUCT(*) and AGGREGATE(*) is very important in the design of Graph Convolutional Network models, which usually focus on designing efficient functions for these two operators.

4 Methodology

We first introduce the framework of SMOG-CF, which can capture high-order connection information in a direct way. Then, we extend our framework to other Graph Convolutional Network based recommender systems, and provide the embedding update formula in matrix form, so that it can be quickly adapted to other basic models. Finally, we present various techniques to improve the model performance.

4.1 Model Framework. Fig.4.1 provides an overview of SMOG-CF, which we describe in detail below.

4.1.1 Raw Input and Embedding Initialization. Our model requires a user-item bipartite interaction graph $\mathcal{G}$ as input. As the raw input of each user (resp. item) is a one-hot vector with a high-dimensional representation, each user and item are embedded into low-dimensional vectors x_u and x_i.

Copyright © 2020 by SIAM
Unauthorized reproduction of this article is prohibited

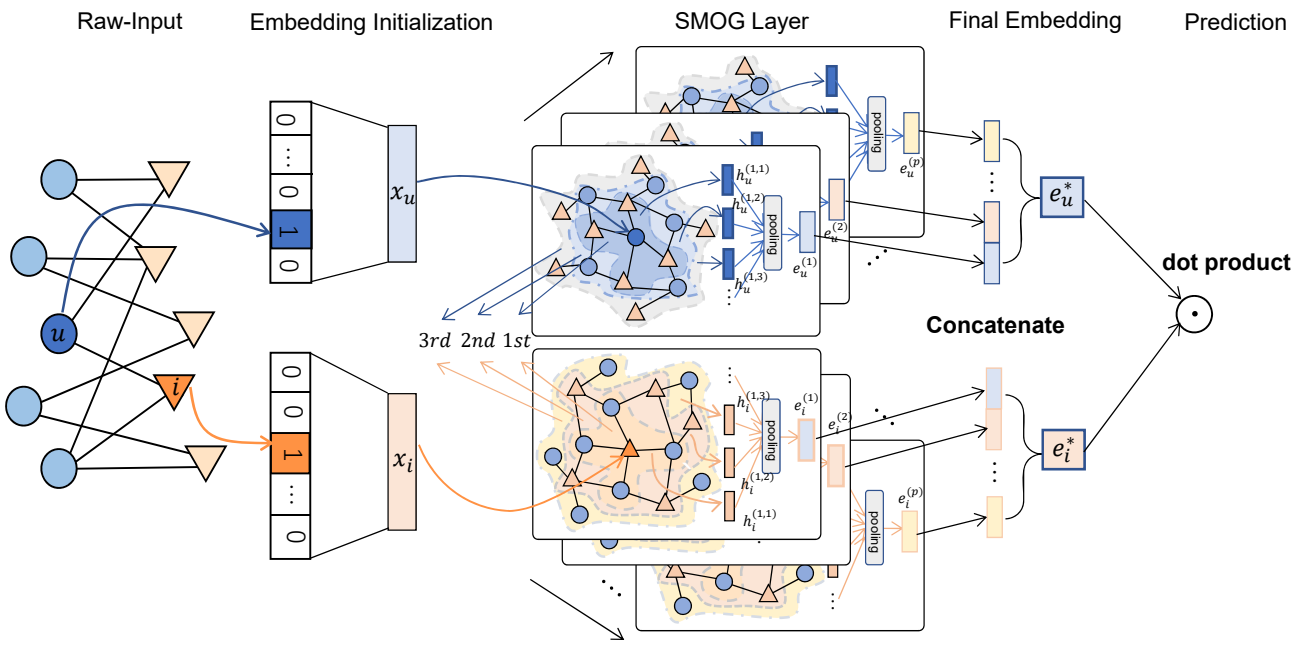

Figure 2: Framework of SMOG-CF.

4.1.2 Stacked Mixed-Order GCN layer. The Stacked Mixed-Order GCNs (SMOG) layer takes the initialized user/item embeddings as input and outputs the final embeddings that take advantage of the information propagated from the neighboring nodes at any order.

In order to better illustrate our method, we first introduce the **path** used for information propagation. Then we present the equations used to calculate the message propagated from a node to its neighboring nodes at any order. Finally, we sum up all the messages passed from a node's neighbors from any distance and present the embedding update formula for each user and item.

Message Construction. Given a node i, node $j \in \mathcal{N}_i$, for any node $k \in \mathcal{N}_j$ (including node i), we define k as the second-order neighbors of i. Similarly, we can define any p-order neighborhood connection: if there exists a connection path from node i to node j through p hops $i \to n_1 \to \cdots \to n_{p-1} \to j$, we call node j a p-order neighboring node of node j, and the path is a p-hop path connecting i and j.

For a p-hop path l_k form node i to node j, following the message construction formula presented by [21], we define the message constructed from j to i through this path as:

$$
\begin{aligned}
m_{ij,k}^{(p)} =& \text{CONSTRUCT}(e_i, e_j, c_{ij}^{(p)}) \\
=& c_{ij,k}^{(p)} \left(W_1^{(p)} e_j + W_2^{(p)} (e_i \odot e_j) \right),
\end{aligned} \tag{4.3}
$$

where W_j^p is the weight vector for distilling the p-order neighboring node j's information and e_i and e_j are the initialized vectors of node i and j respectively. $\odot$ denotes the element-wise product and $c_{ij,k}^{(p)}$ is the connection strength between node i and j through path l_k. Here $c_{ij,k}^{(p)}$ is set as:

$$
c_{ij,k}^{(p)} = \frac{1}{\sqrt{|\mathcal{N}_i|} \ldots |\mathcal{N}_{n_t}| \ldots \sqrt{|\mathcal{N}_j|}}. \tag{4.4}
$$

where $|\mathcal{N}_i|$ denotes the number of first-order neighboring nodes of i. This definition is similar to that in vanilla GCNs [7], but we consider all nodes on the path instead of the end nodes only, so that the connection strength will decay as the length of the path increases.

Message Aggregation. If we use $n_{i \to j}^{(p)}$ to represent the number of paths from node i to its p-order neighboring node j, and $I_{i \leftarrow j}^{(p)}$ to represent the sum of the messages passed from node j to i through all of the p-hop paths, we can conclude that:

$$
I_{i \leftarrow j}^{(p)} = \sum_{k=1}^{n_{i \to j}^{(p)}} m_{ij,k}^{(p)}, \tag{4.5}
$$

Copyright © 2020 by SIAM
Unauthorized reproduction of this article is prohibited

Then we aggregate the messages propagated from all of node i's p-order neighboring nodes:

(4.6)

$$\begin{aligned} I_i^{(p)} &= \sum_{j \in \mathcal{N}_i^{(p)}} \sum_{k=1}^{n_{i \to j}^{(p)}} m_{ij,k}^{(p)} + m_{ii} \\ &= \sum_{j \in \mathcal{N}_i^{(p)} \cup \{i\}} \sum_{k=1}^{n_{i \to j}^{(p)}} c_{ij,k}^{(p)} \left(W_1^{(p)} e_j + W_2^{(p)} (e_i \odot e_j) \right) \\ &= \sum_{j=1}^{N} \sum_{k=1}^{n_{i \to j}^{(p)}} \frac{\hat{A}_{in_{k1}} \dots \hat{A}_{n_{kt} n_{k(t+1)}} \dots \hat{A}_{n_{kp} j}}{\sqrt{D_{ii}} \dots D_{n_{kt} n_{kt}} \dots \sqrt{D_{jj}}} \\ &\quad \left(W_1^{(p)} e_j + W_2^{(p)} e_i \odot e_j \right). \end{aligned}$$

where $N = |\mathcal{U}| + |\mathcal{V}|$ represents the number of nodes in the graph. In the derivation process, we use the equation $|\mathcal{N}_i| = D_{ii}$, where D is the degree matrix of the graph $\mathcal{G}$. Then we transform the formula above into a matrix form

(4.7)

$$\begin{aligned} I_i^{(p)} &= \sum_{j=1}^{N} (\mathbf{D}^{-1/2} \hat{\mathbf{A}} \mathbf{D}^{-1/2})_{i,j}^{p} \left(W_1^{(p)} e_j + W_2^{p} (e_i \odot e_j) \right) \\ &= \left[\left(\mathbf{D}^{-1/2} \hat{\mathbf{A}} \mathbf{D}^{-1/2} \right)^p \left(\mathbf{E} \mathbf{W}_1^{(p)} + (\mathbf{E} \odot \mathbf{E}) \mathbf{W}_2^{(p)} \right) \right]_i . \end{aligned}$$

where the subscript i, j represents the element at the i-th row and j-th column of the matrix, and the subscript i represents the vector at the i-th row. Now we can conclude the matrix form embedding formula by concatenating all nodes' embeddings as a matrix and adopt the ReLU activation:

$$\begin{aligned} \mathbf{H}^{(p)} &= \mathrm{ReLU} \left(\left[I_1^{(p)}, \dots I_t^{(p)} \dots I_1^{(p)} \right] \right) \\ &= \mathrm{ReLU} \left(\hat{\mathcal{L}}^p \left(\mathbf{E} \mathbf{W}_1^{(p)} + (\mathbf{E} \odot \mathbf{E}) \mathbf{W}_2^{(p)} \right) \right), \end{aligned} \tag{4.8}$$

where $\hat{\mathcal{L}} = \mathbf{D}^{-1/2} \hat{\mathbf{A}} \mathbf{D}^{-1/2}$ is the symmetric normalized Laplacian matrix of graph $\mathcal{G}$, $\mathbf{H}^{(p)}$ is the hidden embedding matrix that focuses on the p-order connections between nodes. If we take up to p-order connection into consideration, there would be p different hidden embedding matrixes representing different levels of information. In order to fuse this information together, we devise a pooling operation.

4.1.3 Embedding Pooling. We stack multiple SMOG layers to better exploiting the rich information in the graph. The initialized embedding matrix X could be regarded as the embedding of 0-th layer $E^{(0)}$, then we use the output of the l-the layer as the input of layer $l+1$. Given p hidden embedding matrices from layer l: $\mathbf{H}^{(l,1)}, \cdots, \mathbf{H}^{(l,p)}$, we use Average Pooling, which means using the average of the hidden embedding aggregated from different order nodes as the output embedding of layer l as well as the input of layer $l+1$:

$$\begin{aligned} \mathbf{E}^{(l+1)} &= \mathrm{MEAN}(\mathbf{H}^{(l,1)}, \cdots, \mathbf{H}^{(l,p)}) \\ &= \sum_{k=1}^{p} \mathbf{H}^{(l,k)} / p. \end{aligned} \tag{4.9}$$

Other pooling methods like Max Pooling [9], might also be used. However, in our model, we adopt average pooling because max pooling might focus on low-order hidden embeddings.

4.2 Model Prediction. Following [21], we concatenate the output of each layer in order to enhance the representation capability. For a user u, the representations from l layers are denoted as $\{e_u^{(1)}, \cdots, e_u^{(l)}\}$. As such, we concatenate them to constitute the final embedding for a user; we do the same operation on items, concatenating the item representations learned by different layers to get the final item embedding:

$$e_u^* = \Big\|_{j=1,2,\cdots,l} e_u^{(j)}, \quad e_i^* = \Big\|_{j=1,2,\cdots,l} e_i^{(j)}. \tag{4.10}$$

e_u^* and e_i^* are the final representation of user u and item i respectively. Finally, we take the inner product to estimate the preference of user u toward item i:

$$\hat{y}(u,i) = {e_u^*}^T e_i^*. \tag{4.11}$$

A higher $\hat{y}(u,i)$ indicates that user u is more likely to have an interaction with item i. Other more complex interaction functions such as multi-layer neural networks could also be leveraged to make predictions.

4.3 SMOG-CF as a Framework. As mentioned above, different GCN-based algorithms vary in their design of message construction and aggregation functions. As the proposed SMOG-CF does not depend on a specific basic model, we could extend it to a generalized framework by adopting other message construction and aggregation functions. Here we present the p-order hidden embedding formulas of theses variants in Table 2.

The 'vanilla' column of Table 2 represents the embedding propagation equation from layer l to layer $(l+1)$ while the p-order column represents the p-order hidden embedding $\mathbf{H}^{(l,p)}$ of layer l. Node embeddings of layer $(l+1)$ $\mathbf{E}^{(l)}$ can be calculated using Eq. 4.9. Derivations for other models follow similarly. Noting that the symmetric normalized Laplacian matrix plays

Copyright © 2020 by SIAM
Unauthorized reproduction of this article is prohibited

Table 2: High-Order Embedding propagation formulas for different GCNs

Model	Vanilla formula: $\mathbf{E}^{(l+1)} =$	p-order formula: $\mathbf{H}^{(l+1,p)} =$
GCN [7]	$\text{ReLU}(\hat{\mathcal{L}}\mathbf{E}^{(l)}\mathbf{W}^{(l)})$	$\text{ReLU}(\hat{\mathcal{L}}^p\mathbf{E}^{(l)}\mathbf{W}^{(l,p)})$
GC-MC [19]	$\text{ReLU}\left(\text{ReLU}(\hat{\mathcal{L}}\mathbf{E}^{(l)}\mathbf{W}_1^{(l)})\mathbf{W}_2^{(l)}\right)$	$\text{ReLU}\left(\text{ReLU}(\hat{\mathcal{L}}^p\mathbf{E}^{(l)}\mathbf{W}_1^{(l,p)})\mathbf{W}_2^{(l,p)}\right)$
PinSage [26]	$\text{ReLU}\left(\text{CONCAT}\left(\text{ReLU}(\hat{\mathcal{L}}\mathbf{E}^{(l)}\mathbf{W}_1^{(l)}), \mathbf{E}^{(l)}\right)\mathbf{W}_2^{(l)}\right)$	$\text{ReLU}\left(\text{CONCAT}\left(\text{ReLU}(\hat{\mathcal{L}}^p\mathbf{E}^{(l)}\mathbf{W}_1^{(l,p)}), \mathbf{E}^{(l)}\right)\mathbf{W}_2^{(l,p)}\right)$
NGCF[1][21]	$\text{ReLU}\left(\hat{\mathcal{L}}\left(\mathbf{E}^{(l)}\mathbf{W}_1^{(l)} + (\mathbf{H}^{(l)} \odot \mathbf{H}^{(l)})\mathbf{W}_2^{l}\right)\right)$	$\text{ReLU}\left(\hat{\mathcal{L}}^p\left(\mathbf{E}^{(l)}\mathbf{W}_1^{(l,p)} + (\mathbf{E}^{(l)} \odot \mathbf{E}^{(l)})\mathbf{W}_2^{(l,p)}\right)\right)$

[1] NGCF is the basic model of SMOG-CF.

an important role in the propagation formula (in matrix form), the p-order connection can be easily acquired by $\hat{\mathcal{L}}$ multiplied by itself $p-1$ times, which can be computed efficiently.

4.4 Training Strategies Here we introduce the strategies adopted when training our model.

Loss Function. The loss function used in SMOG-CF is expressed as

$$\mathcal{L} = \mathcal{L}_1 + \lambda\mathcal{L}_2 \tag{4.12}$$

where $\mathcal{L}_1$ is the pair-wise BPR loss function widely used in binary classification problems, and $\mathcal{L}_2$ is the mini-batch aware regularization loss [28]. λ is the trade-off parameter between accuracy and complexity.

$\mathcal{L}_1$ and $\mathcal{L}_2$ can be defined as follows:

$$\mathcal{L}_1 = -\sum_{(u,i,j)\in\mathcal{O}} \log\sigma(\hat{y}_{ui} - \hat{y}_{uj}) \tag{4.13}$$

$$\mathcal{L}_2 = \sum_{(u,i,j)\in\mathcal{O}} \left[\|e_u\| + \|e_i\| + \|e_j\|\right]. \tag{4.14}$$

where $\mathcal{O} = \{(u,i,j)|(u,i) \in \mathcal{R}^+, (u,j) \in \mathcal{R}^-\}$ represents the pairwise training data, $\mathcal{R}^+$ denotes the positive samples while $\mathcal{R}^-$ denotes the negative samples.

Node Dropout. In our model we also adopt dropout to alleviate overfitting. Following prior work on GCNs, we propose to adopt node dropout, which is an extension of dropout in MLP. Specifically, we randomly block a particular node with probability p and discard its outgoing messages. In practice, we randomly drop $(|\mathcal{U}| + |\mathcal{V}|)p$ nodes of the Laplacian matrix.

5 Experiments

We conduct experiments on multiple real-world datasets to evaluate the proposed model. We aim to answer the following research questions:

- **RQ1** How does SMOG-CF perform compared with state-of-the-art collaborative filtering models?
- **RQ2** Is the higher-order connection learned directly in our model necessary for improving performance?
- **RQ3** How do different hyper-parameter settings affect model performance?

5.1 Experiment Setup

5.1.1 Data sets. We apply our model to four datasets: *Gowalla*, *Amazon-Books*, *MovieLens-latest*, and *MovieLens-1m*. All datasets are treated as implicit feedback (i.e., we only consider the presence of an interaction). We provide statistic of the datasets in Table 3. These datasets are summarized as follows:

Table 3: Dataset statistics

Datasets	#Users	#Items	#Clicks	Density
Gowalla	29,858	40,981	1,027,370	0.00084
Amazon-Books	18,343	30,001	805,885	0.00146
Movielens-1m	6,040	3,706	1,000,209	0.04468
MovieLens-latest	610	9,724	100,836	0.01700

Gowalla: This dataset is derived from gowalla[1], a location-based social network where users share their locations by checking-in. We use the cleaned version provided by [21], where only users and items with over 10 interactions are preserved.
Amazon-books: This is a subset of the Amazon-books dataset derived from the Amazon-Review dataset [11].
Movielens-latest & Movielens-1m [5]: These two datasets contain explicit movie rating data collected by GroupLens Research from the Movielens website[2]. The Moivelens-1m dataset is a classical recommendation dataset, while Moivelens-latest a regularly updated version. The version of the Movielens-latest dataset used in our experiments is from September 2018.

5.1.2 Implementation Details We use Tensorflow to implement our model and deploy it on an Nvidia

[1] www.gowalla.com
[2] https://grouplens.org/datasets/movielens/

Copyright © 2020 by SIAM
Unauthorized reproduction of this article is prohibited

Table 4: Experimental Results

	Gowalla		Amazon-books		Movielens-latest		Movielens-1m	
Name	Recall	NDCG	Recall	NDCG	Recall	NDCG	Recall	NDCG
BPR	0.1291	0.1878	0.0261	0.0533	0.2128	0.3384	0.1972	0.3687
NeuMF [6]	0.1326	0.1985	0.0273	0.0551	0.2163	0.3443	0.2009	0.3703
GCN	0.1443	0.2125	0.0301	0.0596	0.2314	0.3607	0.2154	0.3897
GC-MC [19]	0.1480	0.2147	0.0317	0.0606	0.2325	0.3663	0.2187	0.3934
PinSage [26]	0.1491	0.2160	0.0320	0.0615	0.2338	0.3694	0.2195	0.3958
HOP-Rec [24]	0.1499	0.2188	0.0335	0.0642	0.2371	0.3771	0.2235	0.4018
NGCF [21]	0.1547	0.2237	0.0364	0.0672	0.2418	0.3822	0.2277	0.4082
SMOG-CF	**0.1574**	**0.2268**	**0.0383**	**0.0691**	**0.2453**	**0.3874**	**0.2301**	**0.4133**

TiTan X GPU with 11G memory. The batch size B is fixed to 64 for all datasets. The embedding dimension D is set to 64 for Gowalla and Amazon-Books, and 22 for Movielens-latest and Movielens-1m. The learning rate is set to 10^{-4} for Gowalla, $5 * 10^{-4}$ for Amazon-Books and $5 * 10^{-3}$ for Movielens-latest and Movielens-1m. The number of layers and hops is fixed at three if not specified, and the default dropout ratio is 0.2 for all datasets.

5.1.3 Baselines. We choose several comparative methods, including some state-of-the-art models for recommendation, to evaluate the performance of our proposed model. The optimal hyper-parameter settings for each method are determined either by our experiments or as suggested in the original papers.

- BPR [14]: This is a basic model-based recommendation method which uses both user-specific and item-based user embeddings to represent users' preferences.
- NeuMF [6]: This is one of the first models proposed to use deep neural networks to model the interactions between user embeddings and item embeddings. User embeddings and item embeddings are directly learned via back propagation.
- GC-MC [19]: This model considers recommendation as a matrix completion problem, and adopt GCN as a graph auto-encoder framework to generate the representations for users and items.
- PinSage [26]: This model is designed to employ GraphSAGE [4] on item-item graph of pictures on Pinterest[1]. We adopt this model on user-item graph in this paper.
- HOP-Rec [24]: This is a unified method that incorporates factorization with graph-based models. The high-order information is harvest from random surfing among neighborhood items for each user.
- NGCF [21]: This is a recent model that proposed to use Graph Convolution to capture user-item collaborative filtering signals. One contribution of NGCF is that it proposes to stack all layers of GCNs to take high-order user-item connections into consideration.

[1] www.pinterest.com

5.1.4 Evaluation Protocol. We adopt different metrics to evaluate recommendation performance: Recall@k and NDCG@k, two widely used metrics for top-k ranking. By default, we set $k = 20$. The detailed introduction of these metrics are presented in the supplementary material.

5.2 Comparative Results: RQ1. The experimental results of the proposed model and other comparative models are shown in Table 4. Results show that SMOG-CF outperforms other comparative models on these four datasets for top-k recommendation. This is because the proposed SMOG-CF can better exploit high-order connections in user-item graphs through multi-hop paths. Experiments also verify the effectiveness of the proposed model on both medium-scale and small-scale datasets.

5.3 Study of SMOG-CF: RQ2. In order to verify the effectiveness of various components in our model, we conduct an ablation study. As the order of neighbors (or the number of hops taken into consideration) plays an important role, we first study how the performance changes as a function of hop number. In Section 4, we showed that the SMOG-CF framework could be generalized to other GCN models by redefining the message reconstruction and aggregation function, so we extend our SMOG-CF framework to other GCN models

Copyright © 2020 by SIAM
Unauthorized reproduction of this article is prohibited

and would like to study their performance.

5.3.1 Effect of Hop Numbers. To investigate how the order of neighbors affects model performance, we change the depth of neighbors considered in the message propagation step. In particular, we consider hop numbers from 1 to 4. Table 5 summarizes experimental results, where SMOG-CF-p indicates the model considering up to p-order neighbors. In each experiment, we set the number of embedding propagation layers to 3. Note that SMOG-CF-1 is the same as NGCF, as we do not consider high-order connectivity information at each layer, but directly stack all layers together.

Table 5: Effect of hop number.

	Gowalla		Movielens-latest	
Name	Recall	NDCG	Recall	NDCG
SMOG-CF-1	0.1547	0.2237	0.2418	0.3822
SMOG-CF-2	0.1563	0.2256	0.2435	0.3845
SMOG-CF-3	0.1574	0.2268	0.2453	0.3874
SMOG-CF-4	**0.1580**	**0.2276**	**0.2462**	**0.3883**

We find that when we increase the number of hops, SMOG-CF achieves better performance on both datasets. This is because we can directly uncover the information propagated from high-order neighboring nodes at every layer. Another finding is that the marginal improvements decrease as the number of hops increases. This is due to: 1) the farther the distance between two nodes in a graph, the weaker the connection between them. 2) applying a too deep architecture will cause overfitting. Considering the balance between performance and model complexity, we set the hop number as 3 by default.

5.3.2 SMOG-CF as a framework: extension to other GCN models. Here, we apply SMOG-CF to other embedding propagation layers as variants of our models. In particular, we replace the message construction and aggregation functions used in SMOG-CF (the same as that of NGCF, termed SMOG-CF$_{\text{NGCF}}$) with that of GCNs (termed SMOG-CF$_{\text{GCN}}$), PinSage (SMOG-CF$_{\text{PinSage}}$) and GC-MC (SMOG-CF$_{\text{GC-MC}}$). In all experiments, the number of hops and layers is set to three. Results are shown in Table 6.

Experiments show that when extending our architecture to other GCN models, they achieve better performances on both medium and small datasets, compared with the original model performance presented in Table 4. This is because the proposed path-level information propagation for exploiting high-order connections works in different basic GCN models by redefining the information construction and aggregation equations. Also, SMOG-CF$_{\text{NGCF}}$ outperforms other variants as we leverage the direct user-item information interaction as is shown in Eq. 4.3.

Table 6: Performance of different embedding propagation techniques.

	Gowalla		Movielens-latest	
Name	Recall	NDCG	Recall	NDCG
SMOG-CF$_{\text{GCN}}$	0.1493	0.2197	0.2391	0.3798
SMOG-CF$_{\text{GC-MC}}$	0.1533	0.2229	0.2418	0.3831
SMOG-CF$_{\text{PinSag}}$	0.1537	0.2232	0.2427	0.3842
SMOG-CF$_{\text{NGCF}}$	**0.1574**	**0.2268**	**0.2453**	**0.3874**

5.4 Parameter Sensitivity: RQ3. We study the performance variation for our model on the MovieLens-latest dataset with respect to the embedding dimension D. Results are shown in Fig. 3. Our findings are as follows: The embedding dimension plays an important role. When D is too small, the embedding vectors' representation capacity is restricted and cannot fully exploit the abundant information in the user-item graph. When the embedding dimension increases, the performance improves rapidly at the very beginning, but soon the growth slows down and even declines after reaching the best setting due to overfitting.

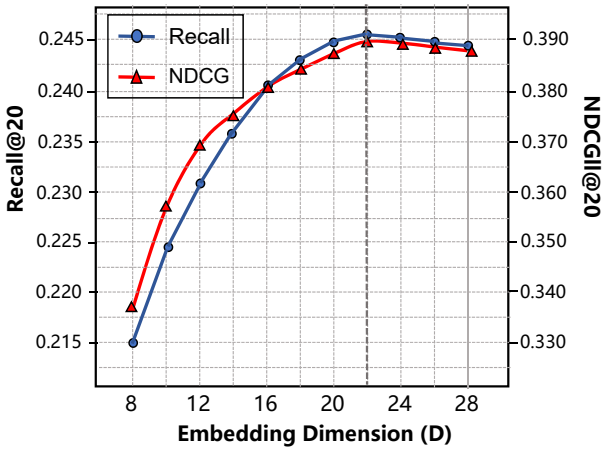

Figure 3: Recall@20 and NDCG@20 of SMOG-CF on MovieLens-latest under different embedding dimension settings. The vertical dotted lines mark the values set in this paper.

6 Conclusion

In this paper, we propose SMOG-CF, a GCN-based model for learning low-dimensional representations of users and items in a recommender system which can

Copyright © 2020 by SIAM
Unauthorized reproduction of this article is prohibited

directly capture higher-order connections in the user-item graph. The crucial point of our method is that we consider information propagation between neighboring nodes at any order by designing path-level information construction and aggregation formulas, and the concise form of the corresponding embedding update formula in matrix form ensures the simplicity and feasibility of our method. We also extended our method to various basic GCN models to make SMOG-CF a general framework. Our comparative experiments and ablation studies showed that our method can learn effective representations of mixed-order user-item collaborative information and thus improve recommendation performance.

As future work, we would like to extend our method to recommender systems with explicit feedback, meaning that the user-item graph would be weighted. We are also interested in applying our method to other graph-structured datasets such as knowledge graph learning.

References

[1] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, H. Harutyunyan, G. V. Steeg, and A. Galstyan, *Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing*, in ICML, 2019, pp. 21–29.

[2] A. Grover and J. Leskovec, *node2vec: Scalable feature learning for networks*, in KDD, 2016, pp. 855–864.

[3] Z. Guo, Y. Zhang, and W. Lu, *Attention guided graph convolutional networks for relation extraction*, in ACL, 2019, pp. 241–251.

[4] W. L. Hamilton, Z. Ying, and J. Leskovec, *Inductive representation learning on large graphs*, in NIPS, 2017, pp. 1024–1034.

[5] F. M. Harper and J. A. Konstan, *The movielens datasets: History and context*, TiiS, 5 (2016), pp. 19:1–19:19.

[6] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T. Chua, *Neural collaborative filtering*, in WWW, 2017, pp. 173–182.

[7] T. N. Kipf and M. Welling, *Semi-supervised classification with graph convolutional networks*, in ICLR, 2017.

[8] J. B. Lee, R. A. Rossi, X. Kong, S. Kim, E. Koh, and A. Rao, *Higher-order graph convolutional networks*, CoRR, abs/1809.07697 (2018).

[9] F. Lei, X. Liu, Q. Dai, B. W. Ling, H. Zhao, and Y. Liu, *Hybrid low-order and higher-order graph convolutional networks*, CoRR, abs/1908.00673 (2019).

[10] Y. Li, D. Tarlow, M. Brockschmidt, and R. S. Zemel, *Gated graph sequence neural networks*, in ICLR, 2016.

[11] J. J. McAuley, C. Targett, Q. Shi, and A. van den Hengel, *Image-based recommendations on styles and substitutes*, in SIGIR, 2015, pp. 43–52.

[12] B. Perozzi, R. Al-Rfou, and S. Skiena, *Deepwalk: online learning of social representations*, in KDD, 2014, pp. 701–710.

[13] J. Qiu, J. Tang, H. Ma, Y. Dong, K. Wang, and J. Tang, *Deepinf: Social influence prediction with deep learning*, in KDD, 2018, pp. 2110–2119.

[14] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, *BPR: bayesian personalized ranking from implicit feedback*, in UAI, 2009, pp. 452–461.

[15] R. A. Rossi, N. K. Ahmed, and E. Koh, *Higher-order network representation learning*, in WWW Companion, 2018, pp. 3–4.

[16] R. A. Rossi, R. Zhou, and N. K. Ahmed, *Estimation of graphlet counts in massive networks*, IEEE Trans. Neural Netw. Learning Syst., 30 (2019), pp. 44–57.

[17] C. Sun, Y. Gong, Y. Wu, M. Gong, D. Jiang, M. Lan, S. Sun, and N. Duan, *Joint type inference on entities and relations via graph convolutional networks*, in ACL, 2019, pp. 1361–1370.

[18] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, *LINE: large-scale information network embedding*, in WWW, 2015, pp. 1067–1077.

[19] R. van den Berg, T. N. Kipf, and M. Welling, *Graph convolutional matrix completion*, CoRR, abs/1706.02263 (2017).

[20] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, *Graph attention networks*, CoRR, abs/1710.10903 (2017).

[21] X. Wang, X. He, M. Wang, F. Feng, and T. Chua, *Neural graph collaborative filtering*, in SIGIR, 2019, pp. 165–174.

[22] Q. Wu, H. Zhang, X. Gao, P. He, P. Weng, H. Gao, and G. Chen, *Dual graph attention networks for deep latent representation of multifaceted social effects in recommender systems*, in WWW, 2019, pp. 2091–2102.

[23] K. Xu, C. Li, Y. Tian, T. Sonobe, K. Kawarabayashi, and S. Jegelka, *Representation learning on graphs with jumping knowledge networks*, in ICML, 2018, pp. 5449–5458.

[24] J. Yang, C. Chen, C. Wang, and M. Tsai, *Hop-rec: high-order proximity for implicit recommendation*, in RecSys, 2018, pp. 140–144.

[25] L. Yao, C. Mao, and Y. Luo, *Graph convolutional networks for text classification*, in AAAI, 2019, pp. 7370–7377.

[26] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, and J. Leskovec, *Graph convolutional neural networks for web-scale recommender systems*, in KDD, 2018, pp. 974–983.

[27] T. Zhang, B. Liu, D. Niu, K. Lai, and Y. Xu, *Multiresolution graph attention networks for relevance matching*, in CIKM, 2018, pp. 933–942.

[28] G. Zhou, X. Zhu, C. Song, Y. Fan, H. Zhu, X. Ma, Y. Yan, J. Jin, H. Li, and K. Gai, *Deep interest network for click-through rate prediction*, in KDD, 2018, pp. 1059–1068.

Copyright © 2020 by SIAM
Unauthorized reproduction of this article is prohibited

Dual Graph Attention Networks for Deep Latent Representation of Multifaceted Social Effects in Recommender Systems

Qitian Wu[1], Hengrui Zhang[1], Xiaofeng Gao[1*], Peng He[2], Paul Weng[3], Han Gao[2], Guihai Chen[1]
[1]Shanghai Key Laboratory of Scalable Computing and Systems,
Department of Computer Science and Engineering, Shanghai Jiao Tong University
[2]WeChat, Tencent Inc.
[3]UM-SJTU Joint Institute, Shanghai Jiao Tong University
{echo740,sqstardust}@sjtu.edu.cn,gao-xf@cs.sjtu.edu.cn,paulhe@tencent.com,
paul.weng@sjtu.edu.cn,alangao@tencent.com,gchen@cs.sjtu.edu.cn

ABSTRACT

Social recommendation leverages social information to solve data sparsity and cold-start problems in traditional collaborative filtering methods. However, most existing models assume that social effects from friend users are static and under the forms of constant weights or fixed constraints. To relax this strong assumption, in this paper, we propose *dual graph attention networks* to collaboratively learn representations for two-fold social effects, where one is modeled by a user-specific attention weight and the other is modeled by a dynamic and context-aware attention weight. We also extend the social effects in user domain to item domain, so that information from related items can be leveraged to further alleviate the data sparsity problem. Furthermore, considering that different social effects in two domains could interact with each other and jointly influence users' preferences for items, we propose a new policy-based fusion strategy based on *contextual multi-armed bandit* to weigh interactions of various social effects. Experiments on one benchmark dataset and a commercial dataset verify the efficacy of the key components in our model. The results show that our model achieves great improvement for recommendation accuracy compared with other state-of-the-art social recommendation methods.

KEYWORDS

Social Recommendation, Graph Attention Network, Social Effect, Representation Learning, Contextual Multi-Armed Bandit

ACM Reference format:
Qitian Wu[1], Hengrui Zhang[1], Xiaofeng Gao[1], Peng He[2], Paul Weng[3], Han Gao[2], Guihai Chen[1]. 2019. Dual Graph Attention Networks for Deep Latent Representation of Multifaceted Social Effects in Recommender Systems. In *Proceedings of Proceedings of the 2019 World Wide Web Conference, San Francisco, CA, USA, May 13–17, 2019 (WWW '19)*, 11 pages.
https://doi.org/10.1145/3308558.3313442

*Xiaofeng Gao is the corresponding author. This work was supported by the National Key R&D Program of China [2018YFB1004703]; the National Natural Science Foundation of China [61872238, 61672353]; the Shanghai Science and Technology Fund [17510740200]; the Huawei Innovation Research Program [HO2018085286]; and the State Key Laboratory of Air Traffic Management System and Technology [SKLATM20180X]; the Tencent Social Ads Rhino-Bird Focused Research Program.

WWW '19, May 13–17, 2019, San Francisco, CA, USA
© Proceedings of the 2019 World Wide Web Conference, published under Creative Commons CC-BY 4.0 License.
ACM ISBN 978-1-4503-6674-8/19/05.
https://doi.org/10.1145/3308558.3313442

1 INTRODUCTION

Recommender systems, which aim at filtering and suggesting items of potential interests to targeted users, have extensive applications and can bring up a huge amount of commercial benefits, especially when the information overload problem has been pervasive with the rapid development of the World Wide Web. Conventional collaborative filtering (CF) methods for recommendation suffer from data sparsity and cold start problems. To this end, social recommendation, which attempts to leverage the social networks of users to incorporate more useful information, has been proposed to alleviate these two problems [5, 12, 18, 19].

Prior Works and Limitations. Nowadays, there are many composite platforms which integrate item recommendation and social network services (SNS). Users on these platforms can click on or rate items as well as interact with their friends. A user's preference for one item could be impacted by her friends, which motivates us to probe into such social effect to improve recommendation quality. Previous studies for social recommendation attempt to model social effects in various ways, such as by trust propagation [5, 12, 21], regularization loss [19, 38], matrix factorization [8, 17, 36], network embedding [16, 32, 41], and deep neural network [4, 31].

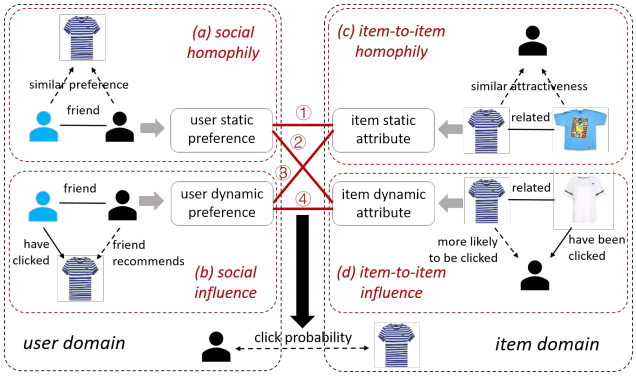

Figure 1: Illustration of the two-fold social effects, i.e., homophily effect and influence effect, in user social networks as well as among related items. The four social effects jointly affect a user's decision on one item.

However, these studies share several common limitations. **First**, most studies assume that linked users all share similar preferences. This assumption cannot suit well contemporary SNS since there

could be various types of online friends, such as close friends, casual friends, and event friends [42]. Therefore, directly equating friendship on an SNS with preference similarity may not be sound. **Second**, most works model friends' influences statically under the forms of constant weights or fixed constraints. This assumption ignores the dynamic pattern of social effects. In fact, users could be influenced by a specific group of friends when it comes to specific items, which makes the social effect dynamic and dependent of specific contexts. **Third**, previous methods lack interpretability for social effects, i.e., they cannot explicitly indicate how a user's preference for one item is influenced by their friends.

Motivations and Rationales. In the real world, people's decisions could be influenced by various factors, and their behaviors are often the results of multifaceted causes. To this end, we probe into four different social effects in recommender systems, including two-fold effects in user domain and the other symmetric two-fold ones in item domain, as illustrated in Fig. 1. In user domain, friends could influence each other in two ways. For one thing, users tend to possess similar preferences to their friends, which is called as *social homophily* [23] (Fig. 1.(a)). Social homophily often leads to intrinsic effect for user preference which stays unchanged and independent of external contexts. For another, a user's friend who has purchased one item may recommend it to that user who then may be more likely to click such item, which is termed as *social influence* [20] (Fig. 1.(b)). Actually, social influence contributes to behavior-level effect for user preference which may change dynamically with specific contexts. *For example, a commercial analyst may be influenced by different communities of friends when it comes to an entertainment article or a commercial news.* Therefore, we can distinguish two components that may impact a user's preference: a static effect from social homophily and a dynamic effect from social influence[1], and we call corresponding user's preference as static and dynamic preference, respectively (left part of Fig. 1).

Moreover, in item domain, there exist analogous 'social effects'. For one thing, some related items tend to possess similar attractiveness when exposed to users. We term this as *item-to-item homophily*, and call the component of an item attribute affected by such effect as static attribute (Fig. 1.(c)). For another, if one item is popular among a certain community, then other related items would become more likely to be clicked by users in this social group. Such phenomenon can be termed as *item-to-item influence*, which depends on specific contexts (Fig. 1.(d)). *For example, a businessman and a researcher both read one article talking about the development of AI, but the businessman would prefer another article about financial information of AI companies, while the researcher would tend to click on another article about cutting-edge breakthroughs in AI.* We call the component of an item attribute affected by such effect as dynamic attribute. Finally, as shown by ①~④ in Fig. 1, the two-fold social effects in both user domain and item domain may jointly influence a user's decision on one item. Such phenomenon is pervasive on Epinion, Twitter, Facebook, and Netflix, etc.

Methodologies and Results. To these ends, in this paper, we propose *DANSER (Dual graph Attention Networks for modeling multifaceted Social Effects in Recommender systems)*, a novel architecture that extends graph attention networks (GAT) [30]. DANSER is composed of two dual GATs: one dual GAT for users—including a GAT to capture social homophily and a GAT to capture social influence—and the other dual GAT for items. Specifically, in user domain, one GAT to capture social homophily attentively aggregates adjacent users' embedding that reflects ones' preferences, and outputs representations for static user preference. Another GAT to capture social influence aims at convolution on adjacent users' context-aware preferences, which characterize relevance of users' rated items to candidate item, and outputs representations for dynamic user preference. In item domain, two analogous GATs as a dual architecture are designed to model items' static and dynamic attributes under effects of item-to-item homophily and influence, respectively, among related items. Such dual mechanism possesses two advantages: i) GATs to capture homophily and influence could collaboratively model two-fold social effects, taking advantage of both global and local views to investigate user-item interactions; ii) two GATs in item domain incorporate information from related items, which could further alleviate data sparsity problem.

Then, since user static (resp. dynamic) preference as well as item static (resp. dynamic) attribute could jointly affect a user's decision on one item (shown by ①~④ in Fig. 1) and the importance of these four interactions would vary for distinct user-item pairs, we propose a policy-based strategy to dynamically weigh the four interactions. Specifically, we model the problem as a contextual multi-armed bandit [1], and treat the weighing strategy as a policy conditional on the context (the targeted user-item pair). Then our goal is to optimize a reward (w.r.t predicted loss). For model training, we use stochastic policy gradient to update our neural network-based policy unit, and design a local-graph aware regularization technique to reduce computational cost for regularization. To verify our model, we conduct experiments on one benchmark datasets from *Epinions* and a commercial dataset from *WeChat Top Story*. The results show that DANSER outperforms state-of-the-art models under both explicit feedback settings (2.9% MAE improvement) and implicit feedback settings (4.5% AUC improvement).

Our contributions can be summarized as follows:

i) **General Aspects:** We distinguish the social homophily and social influence notions in view of static and dynamic effects. Also, we extend the two-fold social effects in user domain to item domain, and therefore investigate four social effects in recommender systems. These general aspects enable the model to capture more information and improve its capacity.

ii) **Novel Methodologies:** We propose DANSER with two dual GATs and a policy-based fusion unit. We are one of the first to use GAT for social recommendation task, and the dual GATs can collaboratively model four social effects in both user and item domains. The policy unit, based on a contextual multi-armed bandit, dynamically weighs four interactions of social effects in two domains according to specific contexts.

[1]In some previous studies, there is little difference between social homophily and social influence. Differently, in this paper, we discriminate these two notions in order to better investigate the dynamic social effect.

iii) **Multifaceted Experiments:** We deploy DANSER on one benchmark dataset and a commercial dataset. The experiment results verify the superiority of DANSER over state-of-the-art techniques, the effectiveness of the proposed components, as well as its good interpretability for modeling social effects.

2 PRELIMINARY AND BACKGROUND

In this paper, we consider a user-item interaction matrix $\mathbf{R} = \{r_{ui}\}_{M\times N}$, where M and N are numbers of users and items respectively. For a recommender system with *implicit* feedback, $\mathbf{R}$ consists of 0 and 1, where $r_{ui} = 1$ if user u has clicked item i, and $r_{ui} = 0$ otherwise. For a recommender system with *explicit* feedback, r_{ui} represents the rating value given by user u on item i (In Epinions, rating values range from 1 to 5 and user could give a high rating if she likes the item), and $r_{ui} = 0$ if user u has not rated item i. We use $R_I(u)$ and $R_U(i)$ to respectively denote the set of items rated by user u and the set of users who have rated item i. Moreover, we assume the existence of a trust or friendship network for users, which is a graph $G_U = (V_U, E_U)$ where V_U is the set of users and E_U is the set of edges that connect two users. We use $F_U(u)$ to denote the set of nodes adjacent to u in G_U. Also, we consider interaction frequencies between user u and user v as edge features, denoted by $\mathbf{e}_{uv} \in \mathbb{R}^C$, where each element in $\mathbf{e}_{uv}$ denotes the frequency of one-type interaction between u and v, and C is the number of interaction types. The edge features reflect link strengths between two users.

Problem Formulation. The social recommendation problem can be defined as [27]: given observed interaction records in $\mathbf{R}$ and trust relationship G_U, we are to estimate unobserved interactions in $\mathbf{R}$, i.e., the probability of a targeted user u^+ clicking an unobserved candidate item i^+ (for implicit feedback) or the rating value user u^+ will score item i^+ (for explicit feedback).

3 METHODOLOGIES

We first introduce the model framework. For each part, we start from the motivations and intuitions, and then introduce the technical details. Secondly, we present some training techniques to improve the model performance. Finally, we discuss two key units in our model for justification.

3.1 Model Framework

Fig. 2 provides an overview of DANSER. We will go into the details of each part in order.

Raw Input and Item Implicit Network. The model requires user-item interaction matrix R and user social network G_U as input. Social recommendation methods focus on leveraging the user social network to solve the data sparsity problem in recommender systems. Most existing methods treat items independently since there is no prior information that explicitly expresses the relationship between items. One way to calculate the similarity or relevance between two items is by the common users who clicked or rated them [25]. For any item i and item j, we define their similarity coefficient s_{ij} as the number of users who clicked both items. These coefficients induce an equivalence relation over items as follows: item i is related to item j if $s_{ij} > \tau$ with τ a fixed threshold. We define the item implicit network as the graph $G_I = (V_I, E_I)$ where V_I is the set of items and E_I is the set of edges that connects two related items.

Embedding Layer. The raw input of each user (resp. item) is a one-hot vector with high dimension, and the embedding operation is to project each user to a low-dimensional representation. Inspired by [15] and [2], each user can be represented by a user-specific embedding and the items rated by her. The former representation reflects user's interests while the latter one (called *item-based user embedding*) captures implicit influence of her rated history on current decision. Such embedding could deal with false-negative samples and improve model accuracy [15]. Similarly, each item can be represented by an item-specific embedding and the users who rated it (called *user-based item embedding*).

For the user-specific embedding, we define $\mathbf{P} = \{\mathbf{p}_u\}_{D\times M}$, where D is the embedding dimension and $\mathbf{p}_u$ denotes the embedding vector for user u. For the item-based user embedding, we define another embedding lookup $\mathbf{Y} = \{\mathbf{y}_i\}_{D\times N}$, where $\mathbf{y}_i$ denotes the item latent factor. Then each item in $R_I(u)$ will be mapped via $\mathbf{Y}$ and outputs $\mathcal{Y}_u = \{\mathbf{y}_j | j \in R_I(u)\}$, which will be used to compute the item-based user embedding, a D-dimensional vector representation, in the next layer. The user embeddings reflect a user's inherent preference (the 'inherent' notion emphasizes that it contains no social information and only depends on the user oneself). Similarly, for the item-specific embedding, we define $\mathbf{Q} = \{\mathbf{q}_i\}_{D\times N}$, where $\mathbf{q}_i$ denotes the embedding vector for item i. For user-based item embedding, we define $\mathbf{X} = \{\mathbf{x}_u\}_{D\times M}$, where $\mathbf{x}_u$ is user latent factor, and each user in $R_U(i)$ will be mapped via $\mathbf{X}$ and outputs $\mathcal{X}_i = \{\mathbf{x}_v | v \in R_U(i)\}$, which will be used to compute user-based item embedding, a D-dimensional vector representation, in the next layer. Also, the item embeddings reflect item's inherent attribute.

Dual GCN/GAT Layer. Graph convolution network (GCN) [14] conducts local convolutional operation over neighbor nodes in graph and outputs a new representation for each node, in order to encode the graph structure information as low-dimensional node representations. Such operation can be viewed as an extension of convolution neural network (CNN) from a grid structure to general graphs. GCN equally aggregates the neighbors' embedding in each convolution and treats each neighbor nodes with equal importance. In contrast, Graph Attention Network (GAT) [30] leverages attention mechanism to consider different weights from neighbor nodes, which enables the model to filter out noises and focus on important adjacent nodes.

In order to exploit user social network G_U and item implicit network G_I, we propose dual graph attention networks, an extension of GCN/GAT as a means to non-linearly aggregate the embeddings of the neighbor nodes in both networks, so that the social effects from friends and related items can propagate to adjacent nodes through the graphs. As discussed in Section 1, we can distinguish two types of social effects for users (resp. items) in social recommender systems: *homophily* effect and *influence* effect. They jointly affect user preferences and item attributes in different ways. DANSER uses two dual GATs to collaboratively learn different deep representations for user static/dynamic preference and item static/dynamic attribute.

I. GAT to capture social homophily (marked as blue in Fig. 2). Firstly, via user embedding, we have $\mathbf{P}$ as representation of inherent user preference factor. Then the GCN/GAT operation could output

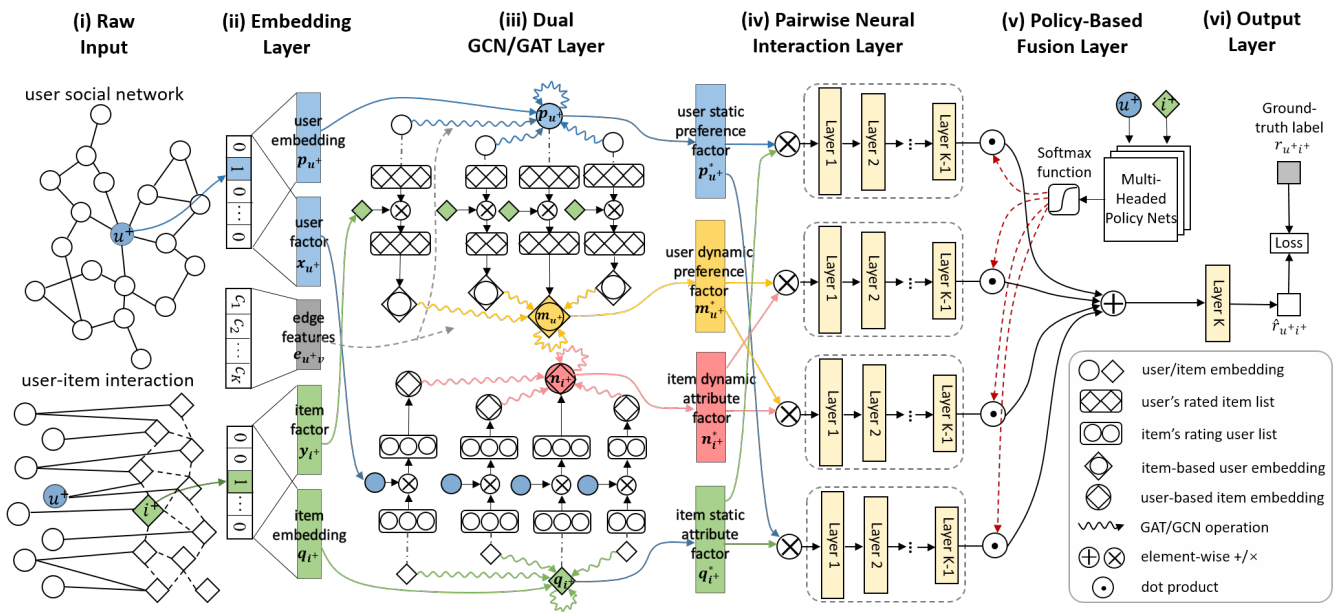

Figure 2: DANSER framework. Blue circles denote a targeted user u^+ while green rhombuses denote a candidate item i^+. i) The model requires user-item interaction records and user social network as raw input. We use common users who rate both items to calculate the item-item relevance and link the related items to form an item implicit network (the dotted lines between items). ii) In the embedding layer, we represent one user (resp. item) as a low-dimensional embedding vector and a latent factor. Besides, interaction frequencies between users are used as edge features. iii) In the dual GCN/GAT layer, four different graph attention networks are to capture the two two-fold social effects, where the upper (resp. lower) two of them output representations for user (resp. item) static and dynamic preferences (resp. attributes) under the effect of homophily and influence, respectively. iv) These four deep factors will be pairwisely combined as four interacted features, which are then fed into four independent neural networks to obtain more condensed representations. v) Then a policy net with the input of item i^+'s and user u^+'s embeddings as context information outputs weights for four interacted features, which will be aggregated as one synthetic vector. vi) Finally, the synthetic vector is input into the output layer to give the final predicted score $\hat{r}_{u^+i^+}$.

a new representation, the user static preference factor $\mathbf{P}^*$, by

$$\mathbf{P}^* = \sigma(\mathbf{A}_P(G_U)\mathbf{P}\mathbf{W}_P^T + \mathbf{b}_P), \tag{1}$$

where $\sigma, \mathbf{W}_P, \mathbf{b}_P$ are activation function, weight matrix, bias vector respectively, and $\mathbf{P}^*$ is the updated representation of users, which incorporates social information by using the attention weights $\mathbf{A}_P(G_U) = \{\alpha^P_{uv}\}_{M\times M}$ obtained from G_U. Its elements are defined as follows:

$$\alpha^P_{uv} = \frac{attn_U(\mathbf{W}_P\mathbf{p}_u, \mathbf{W}_P\mathbf{p}_v, \mathbf{W}_E\mathbf{e}_{uv})}{\sum_{w\in\Gamma_U(u)} attn_U(\mathbf{W}_P\mathbf{p}_u, \mathbf{W}_P\mathbf{p}_w, \mathbf{W}_E\mathbf{e}_{uv})}, v \in \Gamma_U(u),$$

where $\mathbf{W}_E$ is weight matrix, $\Gamma_U(u) = \{u\} \cup F(u)$, $attn_U(x, y, z) = LeakyReLu(\mathbf{w}_U^T z \otimes (x||y))$, and $\mathbf{w}_U$ is weight vector. Here $\otimes$ and $||$ denote the element-wise product and concatenation, respectively. Note that the above GAT weight α^P_{uv} remains unchanged given user u and v, which means factor matrix $\mathbf{P}^*$ is fixed for users. This design conforms to the intuition that social homophily contributes to static effect for user's preference, this is why we call it static preference factor.

II. GAT to capture social influence (marked as yellow in Fig. 2). In contrast to social homophily, social influence effect is often context-aware, and the model needs to output different attention weights for friends w.r.t different candidate items. Through the embedding layer, we have $\mathcal{Y}_u$ as the embedding of the items clicked by user u. Then we let each item clicked by user u interact with candidate item i^+,

$$\mathcal{Y}_u^{i^+} = \{\mathbf{y}_j \otimes \mathbf{y}_{i^+} | j \in R_I(u)\}.$$

This product operation can help to focus on the candidate item and model dynamic social influence under a specific context. We define the item-based user embedding $\mathbf{M}_{i^+} = \{\mathbf{m}_u^{i^+}\}_{D\times M}$, which depends on candidate item i^+ with max pooling to select the most dominating features for D dimensions:

$$m_{ud}^{i^+} = \max_{j\in R_I(u)} \{y_{jd} \cdot y_{i^+d}\} \quad \forall d = 1, \ldots, D$$

where $m_{ud}^{i^+}$, y_{i^+d}, and y_{jd} are the d-th feature of $\mathbf{m}_u^{i^+}$, $\mathbf{y}_{i^+}$, $\mathbf{y}_j$ respectively. The max pooling operation can help to focus on the most important value and alleviate noises in users' clicked history. The item-based embedding $\mathbf{m}_u^{i^+}$ includes two information: i) user u's context-aware preference (w.r.t candidate item i^+), and ii) inherent representation for u (independent of social information). In order to define the user dynamic preference factor $\mathbf{M}^*_{i^+}$, we proceed to incorporate the social information from friend users,

$$\mathbf{M}^*_{i^+} = \sigma(\mathbf{A}_M(G_U)\mathbf{M}\mathbf{W}_M^T + \mathbf{b}_M), \mathbf{A}_M(G_U) = \{\alpha^M_{uv,i^+}\}_{M\times M},$$

$$\alpha^M_{uv,i^+} = \frac{attn_U(\mathbf{W}_M\mathbf{m}_u^{i^+}, \mathbf{W}_M\mathbf{m}_v^{i^+}, \mathbf{W}_E\mathbf{e}_{uv})}{\sum_{w\in\Gamma_U(u)} attn_U(\mathbf{W}_M\mathbf{m}_u^{i^+}, \mathbf{W}_M\mathbf{m}_w^{i^+}, \mathbf{W}_E\mathbf{e}_{uv})},$$

for $v \in \Gamma_U(u)$. Note that the above attention weight α^M_{uv,i^+} depends on the user's history of rated items as well as specific candidate item i^+, which indicates that factor matrix $\mathbf{M}^*_{i^+}$ would change dynamically with different contexts. This design conforms to the intuition that social influence contributes to context-aware effect for a user's preference, so we term it as dynamic preference factor.

III. GAT to capture item-to-item homophily (marked as green in Fig. 2). Similarly, we use the item embedding $\mathbf{Q}$ as representation of inherent item attribute factor, and then leverage GAT to incorporate social information,

$$\mathbf{Q}^* = \sigma(\mathbf{A}_Q(G_I)\mathbf{Q}\mathbf{W}_Q^T + \mathbf{b}_Q), \mathbf{A}_Q(G_I) = \{\alpha^Q_{ij}\}_{N\times N},$$

$$\alpha^Q_{ij} = \frac{attn_I(\mathbf{W}_Q\mathbf{q}_i, \mathbf{W}_Q\mathbf{q}_j)}{\sum_{k\in\Gamma_I(i)} attn_I(\mathbf{W}_Q\mathbf{q}_i, \mathbf{W}_Q\mathbf{q}_k)}, j \in \Gamma_I(i),$$

where $attn_I(x, y) = LeakyRelu(\mathbf{w}_I^T(x||y))$, and $\mathbf{w}_I$ is a weight vector. The GAT weight α^Q_{ij} remains unchanged given item i and j. Correspondingly, item-to-item homophily contributes to static effect for item's attribute, and we call it static attribute factor.

IV. *GAT to capture item-to-item influence (marked as red in Fig. 2).* We now model item-to-item influence, which is context-aware and depends on the specific targeted user. Hence, our model needs to output different attention weights for distinct related items w.r.t different targeted users. Similarly to social influence, the user-based item representation $N_{u^+} = (n_i^{u^+})_{D\times N}$ for a given targeted user u^+ can be defined as follows:

$$\mathcal{X}_i^{u^+} = \{\mathbf{x}_v \otimes \mathbf{x}_{u^+} | v \in R_I(i)\},$$

$$n_{id}^{u^+} = \max_{v\in R_U(i)} \{x_{vd} \cdot x_{u^+d}\} \quad \forall d = 1, \ldots, D,$$

where $n_{id}^{u^+}$, x_{u^+d}, and x_{vd} are the d-th feature of $\mathbf{n}_i^{u^+}$, $\mathbf{x}_{u^+}$, $\mathbf{x}_v$ respectively. Then the representation for item dynamic attribute $\mathbf{N}^*_{u^+}$ can be computed as follows:

$$\mathbf{N}^*_{u^+} = \sigma(\mathbf{A}_N(G_U)\mathbf{N}_{u^+}\mathbf{W}_N^T + \mathbf{b}_N), \mathbf{A}_N(G_U) = \{\alpha^N_{ij,u^+}\}_{N\times N},$$

$$\alpha^N_{ij,u^+} = \frac{attn(\mathbf{W}_N\mathbf{n}_i^{u^+}, \mathbf{W}_N\mathbf{n}_j^{u^+})}{\sum_{k\in\Gamma_I(i)} attn(\mathbf{W}_N\mathbf{n}_i^{u^+}, \mathbf{W}_N\mathbf{n}_k^{u^+})}, j \in \Gamma_I(i).$$

Pairwise Neural Interaction Layer. Since user's decision on one item often depends on both user preference and item attribute, we can let two preference factors and two attribute factors pairwisely interact with each other. Then we borrow the idea from [11], feeding the four results into different neural networks indexed by $a \in \{1, 2, 3, 4\}$:

$$\mathbf{s}_a = \phi_K^a(\cdots\phi_2^a(\phi_1^a(\mathbf{z}_0[a]))),$$

$$\phi_k^a(\mathbf{z}_{k-1}) = tanh(\mathbf{W}_k^a\mathbf{z}_{k-1}^a + \mathbf{b}_k^a), k \in [1, K-1],$$

$$\mathbf{z}_0 = [\mathbf{p}_u^* \oplus \mathbf{q}_i^*, \mathbf{p}_u^* \oplus \mathbf{n}_i^*, \mathbf{m}_u^* \oplus \mathbf{q}_i^*, \mathbf{m}_u^* \oplus \mathbf{n}_i^*],$$

Here we employ a tower structure for each network, where higher layers have smaller number of neurons [11].

Policy-Based Fusion Layer. We further fuse the four interacted features $\mathbf{s}_a$ into a synthetic one. Note that homophily effect and influence effect could jointly affect user preference and item attribute, but for distinct users and items, the importance of the two-fold social effects could be different. To this end, we propose a new policy-based fusion strategy to dynamically allocate weights to the four interacted features according to specific user-item pairs.

We model the weight allocation as a contextual multi-armed bandit problem[1],where an action, denoted by $\gamma \in \{1, 2, 3, 4\}$, indicates which feature to choose, a context is a user-item pair and the reward after playing an action represents a recommendation loss. In this problem, a stochastic policy can be written as the conditional probability $p(\gamma|\mathbf{p}_u, \mathbf{q}_i)$, which could characterize the selecting probability or importance weights for different combinations of social effects given a specific pair (u, i).

To make the problem solvable, we approximate $p(\gamma|\mathbf{p}_u, \mathbf{q}_i)$ by a neural network (called policy network):

$$e_\gamma = \mathbf{W}_F^2 \tanh(\mathbf{W}_F^1(\mathbf{p}_u||\mathbf{q}_i) + \mathbf{b}_F^1) + \mathbf{b}_F^2.$$

$$p(\gamma|\mathbf{p}_u, \mathbf{q}_i) = \frac{\exp(e_\gamma)}{\sum_{a=1}^4 \exp(e_a)}.$$

Then the synthetic representation can be expressed as

$$\mathbf{s} = \mathbb{E}_{\gamma\sim p(\gamma|\mathbf{p}_u,\mathbf{q}_i)}(\mathbf{s}_\gamma) = \sum_{\gamma=1}^4 p(\gamma|\mathbf{p}_u, \mathbf{q}_i) \cdot \mathbf{s}_\gamma.$$

We call the above strategy *single-headed* policy-based fusion. Inspired by [29], we can extend it to a *multi-headed* version. We harness L different independent policy networks and the final weights are given by the averaged weights given by each policy net. The training of policy networks is by stochastic policy gradient, which will be discussed in Section 3.2.

Output Layer. Then probability of user u clicking item i can be predicted by

$$\hat{r}_{ui} = nn(\mathbf{s}).$$

If a clicking probability is required (for implicit feedback), $nn(\cdot)$ can be a fully-connected layer with a sigmoid activation function. If a rating value is needed (for explicit feedback), $nn(\cdot)$ can be a fully-connected layer without activation function.

Loss Function. The loss function measures the discrepancy between predicted value and ground-truth value. For implicit feedback, the most widely adopted loss function is the cross-entropy defined as

$$\mathcal{L}_1 = -\sum_{(u,i)} r_{ui} \log \hat{r}_{ui} + (1 - r_{ui}) \log(1 - \hat{r}_{ui}).$$

For explicit feedback, we adopt the mean square loss

$$\mathcal{L}_1 = \sum_{(u,i)} \|\hat{r}_{ui} - r_{ui}\|^2.$$

Here, we concentrate on pointwise loss. Also, other pairwise loss functions can be used and we leave them for future investigation.

3.2 Training

In this subsection, we introduce some training techniques to improve the model performance.

Mini-Batch Training. We also leverage mini-batch training to calculate the gradient. For each iteration, we consider B user-item pairs. For each pair (u, i), the neighbor nodes $F_U(u)$, $F_I(i)$ in two networks as well as the rated history $R_I(v)$, $v \in \Gamma_U(u)$, and $R_U(j)$, $j \in \Gamma_I(i)$ will be together input into the model to compute the gradient. Once an epoch is finished, the batch partition will be randomly reset to incorporate enough noise during optimization. In each mini-batch, since numbers of friends are different for users, we

need to pad zeros and input a matrix with dimension of max friend number in one batch. We observe that number of friends tends to obey a long-tail distribution, so the padding would lead to both high space and time complexity. Therefore, we use a sample technique: 1) for friend number more than F, we uniformly sample F friends as input data; 2) for friend number less than F, we pad zeros to obtain a F-dimensional vector. We also experiment other sample methods like random walk with restart (RWR) [28] on a sub-graph near central node, but we find that uniformly sampling over adjacent nodes can achieve competitive performance. This observation could be justified by one argument in sociology research that influence from users always propagates to adjacent users in social networks [7]. Thus we adopt the simpler method to reduce time complexity. In experiment part, we will investigate the impact of sample size F on the performance.

Local-Graph Aware Regularization. The other issue is regularization to avoid over-fitting problem. In our model, we adopt L1 regularization to constrain the embedding parameters to sparse forms. The regularization loss can be expressed as

$$\mathcal{L}_2 = \sum_u (\|p_u\| + \|x_u\|) + \sum_i (\|q_i\| + \|y_i\|).$$

However, the above loss introduces high computational complexity, since for each batch training we need to calculate the gradient of regularization over all users and items. Such operation can not generalize to large-scale data set. A recent work [43] proposes a mini-batch aware regularization trick to solve this problem. In each batch training, only the embedding parameters for user-item pair (u, i) in the batch will be used to calculate the regularization loss. Here we extend this idea to graph structure and call it as local-graph aware regularization. In each batch training, we constrain the embedding parameters of users, items as well as their neighbor nodes in social network and item implicit network. The new regularization loss can be written as

$$\mathcal{L}_2 = \frac{1}{2} \sum_{(u,i)} \Big[\|\mathbf{p}_u\| + \|\mathbf{x}_u\| + \sum_{v \in \Gamma_U(u)} \frac{1}{|F_U(v)|} (\|\mathbf{p}_v\| + \|\mathbf{x}_v\|) + \|\mathbf{q}_i\| + \|\mathbf{y}_i\| + \sum_{j \in \Gamma_I(i)} \frac{1}{|F_I(j)|} (\|\mathbf{q}_j\| + \|\mathbf{y}_j\|) \Big].$$

In fact, the local-graph aware regularization is to split the regularization loss into small parts for each mini-batch and guarantees that once an epoch is finished, each user and each item can be regularized equally. To sum up, the final loss function is

$$\mathcal{L} = \mathcal{L}_1 + \lambda \mathcal{L}_2, \tag{2}$$

where λ is a trade-off parameter between accuracy and complexity.

Besides, for parameters in neural networks (weight matrix and bias vectors), we adopt dropout strategy to replace the traditional regularization. Our experiments will further discuss the influence of different dropout probabilities ρ.

Policy Gradient. In training stage, we train the policy networks in a stochastic way. Assume $p_l(\gamma|\mathbf{p}_u, \mathbf{q}_i)$ to be the active probability given by l-th policy network, and random variable γ obeys a multinomial distribution. For each policy network, we draw $\gamma \sim Multi(p_l(\gamma|\mathbf{p}_u, \mathbf{q}_i))$, and feed $\mathbf{s}_\gamma$ into output layer, which gives prediction and returns loss. We leverage the loss to train the policy network. This design can be formulated as an actor-critic terminology: i) given the environment (user u and item i), the l-th agent (policy net) needs to execute an action according to policy $(p_l(\gamma|\mathbf{p}_u, \mathbf{q}_i))$; ii) every action would generate a reward (recommendation loss) that could lead the training of policy network. We define the reward as subtraction of loss generated by selected interacted feature, i.e., $\mathcal{R}(p_u, q_i, \gamma) = -\mathcal{L}(\mathbf{s}_\gamma)$ (where $\mathcal{L}(\mathbf{s}_\gamma)$ denotes the loss when we input $\mathbf{s}_\gamma$ into the output layer). We aim at maximizing the expected reward $\mathbb{E}_{\gamma \sim p(\gamma|p_u, q_i)}(\mathcal{R}(p_u, q_i, \gamma))$. The policy gradient method REINFORCE [33] can be used to update parameters in one policy network (denoted as θ), and the gradient can be derived as follows:

$$\begin{aligned} &\nabla_\theta \mathbb{E}_{\gamma \sim p_\theta(\gamma|\mathbf{p}_u, \mathbf{q}_i)}(\mathcal{R}(\mathbf{p}_u, \mathbf{q}_i, \gamma)) \\ &\simeq \frac{1}{4} \sum_\gamma \nabla_\theta \log p_\theta(\gamma|\mathbf{p}_u, \mathbf{q}_i) \mathcal{R}(\mathbf{p}_u, \mathbf{q}_i, \gamma). \end{aligned} \tag{3}$$

Training Algorithm. The training of DANSER updates policy networks and feedforward networks (including embedding, dual GAT, pairwise neural interaction and output layers) iteratively, which is presented in Alg. 1. First, we update feedforward networks for n_p steps (one mini-batch for one step) by optimizing (2). Then we train L policy networks independently by conducting corresponding policy gradient (3).

Algorithm 1: Training for DANSER

REQUIRE: $\mathbf{R}$, observed user-item interaction. G_U, user social network. G_I, item implicit network.
REQUIRE: η, ζ, learning rates.
while *not converged* **do**
  for $i = 1, \cdots, n_p$ **do**
    Sample B user-item pairs.;
    For each pair (u, i), uniformly sample F neighbors from $F_U(u)$ and $F_I(i)$, respectively;
    Draw $\gamma_l \sim Multi(p^l_\theta{}_l|\mathbf{p}_u, \mathbf{q}_i)$, $l = 1, \cdots, L$;
    $w \leftarrow \eta \nabla_w \sum_l L(\mathbf{s}_{\gamma_l})$;
  $\theta_l \leftarrow \zeta \nabla_{\theta_l} \mathbb{E}_{\gamma \sim p^l_{\theta_l}(\gamma|\mathbf{p}_u, \mathbf{q}_i)}(\mathcal{R}(\mathbf{p}_u, \mathbf{q}_i, \gamma))$, $l = 1, \cdots, L$;

Complexity Analysis. Since complexity and scalability are two important concerns for graph algorithm, we investigate complexity of DANSER for model training and inference respectively. For training, we need to update embedding of all friend users and related items for B user-item pairs in one batch, which requires time complexity $O(BFCD)$, where $C = \max\{|R_I(v)||v \in \Gamma_U(u)\} \cup \{|R_U(j)||j \in \Gamma_I(i)\}$. We can control C to a limited value by truncating recent C_t rated items for users. The friend number F is controlled by our sample method. Also, for inference, we need $O(BFCD)$ to compute GAT representations. Hence, DANSER can scale linearly w.r.t number of user-item interactions.

3.3 Discussions

In this subsection, we discuss on effectiveness of our key units, dual GAT layer and policy-based fusion layer, in order to shed more insights on DANSER.

Justification of Dual GATs. During the training process, the GAT to capture the homophily effect would give larger weights

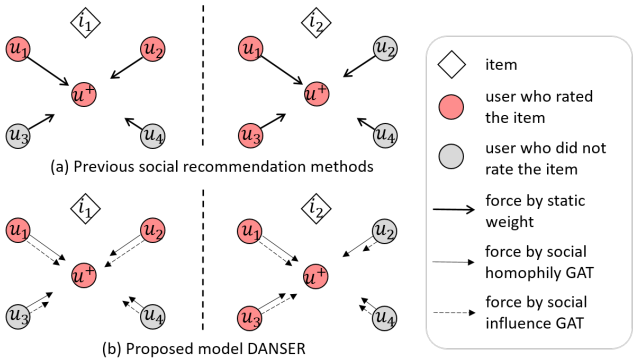

Figure 3: Illustration of effectiveness of dual GATs and comparison with previous social recommendation methods.

to friends whose preferences for items are globally similar to the targeted user (resp. related items whose attractiveness to users are globally similar to candidate item), so the static preference (resp. attribute) factor captures static effect of friend users (resp. related items) from a global view. Differently, the weights given by the GAT to capture the influence effect depend on the targeted user (resp. item), so the corresponding dynamic factor captures context-aware effects from a local view. In each domain, a dual GAT collaboratively models the two-fold effects and takes both global and local views to conduct a more accurate investigations into distinct users and items.

To put it more symbolically, we illustrate its effectiveness by considering the following two cases for the recommendation of candidate item i^+ to targeted user u^+: item i^+ is similar to item i_1 and i^+ is similar to i_2 (see left and right columns of Fig. 3). Here, we assume that user u^+ has only clicked on i_1 and i_2 (and no other items), and users u_1 to u_4 are u^+'s friends. Previous works for social recommendation consider static effect of friends in an average manner. As illustrated in Fig. 3.(a), for friend u_1 who clicked on both items, the attention weight would be large, and the opposite for friend u_4. For u_2 and u_3, their weights are medium since they are both partially similar to u^+. Therefore static weights cannot discriminate between the influence of user u_2 and u_3 when it comes to item i^+ even if it is known that it is similar to i_1 or i_2. In contrast, DANSER contains two different attention weights, where one captures user-specific static effect (for homophily) and the other can dynamically change under different contexts (i.e., for influence). When candidate item i^+ is similar to i_1 (resp. i_2), the social influence GAT would focus more on u_1 and u_2 (resp. u_1 and u_3). Then the resultant force of the two GATs would make u^+ pay more attention to u_2 when i^+ is similar to i_1 and to u_3 in the other case. The global and local views of DANSER work collaboratively for selecting the 'right' neighbors that are similar to the targeted user under a specific context, which improves model expressiveness. In Section 4.5, we will verify the above argument via a real-case study.

Justification of Policy-Based Fusion. The policy-based fusion layer acts like a stochastic network while training, and the REINFORCE algorithm can help to explore the optimal weights distribution over four interacted features. Since the interacted feature with larger active probability is more likely to be selected, it would be more frequent for the interaction neural layer that gives the 'best' feature to be trained. Such mechanism brings up two major benefits. Firstly, the policy networks can filter 'right' user-item samples for training of corresponding interaction neural layer as well as previous GCN/GAT and embedding layers. Second, the active probability introduces random pattern to the training process, which can help to escape from local optimums. Our ablation study in experiment part also validate the superiority of the policy-based fusion compared with other straightforward strategies.

4 EXPERIMENTS

To comprehensively evaluate proposed model DANSER[2], we conduct experiments to answer the following research questions:
RQ1 How does DANSER perform compared with state-of-the-art models for recommendation and social recommendation?
RQ2 Are the key components in DANSER, such as dual GATs and policy-based fusion, necessary for improving performance?
RQ3 How do hyper-parameters in DANSER impact recommendation performance?
RQ4 How can DANSER interpret the four different social effects from real-world data?

4.1 Experiment Setup

4.1.1 Data sets. We apply our model to one public benchmark datasets for social recommendation *Epinions* and a commercial dataset *WeChat Top Story*. We provide some statistics of these two datasets in Table 1. The basic information about the two datasets is summarized as follows:

i) *Epinions:* Epinions is a consumer review websites, where users can rate some items and add other users in their trust lists. *Epinions* dataset [22] contains two parts of information: user-item interaction pairs, where items are rated from 1 to 5 (explicit feedback), as well as the directed trust relationships between users. The dataset has been widely used as benchmarks for social recommendation. Following other works [10, 16, 31, 36], we randomly choose 80% user-item interactions as training set and the remaining 20% as test set for each dataset.

ii) *WeChat:* We also deploy our model on a real-world article recommender system, WeChat Top Story. This dataset is constructed by user-article clicking records on this platform. Different from *Epinions*, this dataset only contains implicit feedback, i.e., we only know whether a user clicked an article or not. Besides, the friend relationships are bidirectional. We chronologically order the user-article clicking records, and use the first 90% records to train our model and the remaining records to evaluate the prediction performance. Since the positive samples and negative samples are fairly unbalanced in this dataset, we uniformly sample the negative ones such that the numbers of positive and negative samples are the same for one user.

Table 1: Statistics of three datasets.

Dataset	#users	#items	#interactions	#relationships
Epinions	49,290	139,738	664,824	487,181
WeChat	~200,000	~100,000	~4,000,000	~2,000,000

[2]The codes are released at https://github.com/echo740/DANSER-WWW-19.

4.1.2 Implementation Details. We use Tensorflow to implement our model and deploy it on GTX 1080 GPU with 8G memory. The hyper-parameter settings for *Epinions* are as follows: batch size $B = 64$, embedding dimension $D = 10$, dropout ratio $\tau = 0.5$, regularization coefficient $\lambda = 0.001$, sample size $F = 30$, truncation length $C_t = 30$, multi-head number $L = 4$, policy gradient period $n_p = 1000$. The learning rates $\eta = 0.1$, $\zeta = 0.01$. The Leaky ReLu slope is 0.2. In pairwise interaction neural layer, we adopt three-layer neural networks and neuron numbers are $10 - 16 - 8 - 4$.

4.1.3 Competitors. We choose several comparative methods, including some state-of-the-art models for recommendation and social recommendation, to evaluate the performance of our DANSER. The optimal hyper-parameter settings for each method are determined either by our experiments or suggested by previous works.

The recommendation models include *SVD++*[15] and *DELF*[2]. These two methods only leverage user-item interactions as model input. They can be used to evaluate the effectiveness of other models that use social information.

- *SVD++* [15] is a basic model-based recommendation method which use both user-specific and item-based user embedding to represent user's preferences.
- *DELF* [2] is a state-of-the-art CF method which proposes dual embedding for users and items, and adopts deep neural networks to capture complex non-linear information.

The social recommendation models include *TrustPro*, *TrustMF*, *TrustSVD*, *NSCR*, *SREPS*. These methods all leverage both user-item interactions and user friendships as input information.

- *TrustPro* [37] is a trust propagation method, using rating of friends to deduce rating of targeted user.
- *TrustMF* [36], as one matrix factorization method, optimizes user embedding to retrieve the trust matrix.
- *TrustSVD* [10] is another matrix factorization-based method, incorporating friends' embedding vectors into targeted user's predicted rating.
- *NSCR* [31], as a strong baseline for social recommendation, adopts deep neural networks to learn latent representations of users and items, and leverages graph regularization to constrain the embedding of adjacent users to be similar.
- *SREPS* [16] is another strong baseline, using a network embedding approach to encode social network.

We also construct several variants of DANSER as ablation study. To evaluate the necessity of dual GAT layer, we simplify our model as *DualEMB*, *DualGCN*, *userGAT*, and *itemGAT*. *DualEMB* removes the convolution over adjacent nodes, while *DualGCN* replace GAT by GCN. *userGAT* removes two GATs in item domain, and *itemGAT* removes two GATs in user domain. Moreover, to evaluate the necessity of policy-based fusion layer, we consider the following variants. *DANSER-m* and *DANSER-a* leverage max and average pooling to fuse four interacted features, respectively. *DANSER-c* concatenates the four interacted features as the input of output layer. *DANSER-w* removes the policy net, set four sharing fusion weights, and update the weights together with training of feed-forward networks.

4.1.4 Evaluation Protocol. We adopt different metrics to evaluate recommendation performance. Since two datasets possess different feedbacks, we consider different metrics for them. For *Epinions* with explicit feedback, we use *MAE* and *RMSE*, which are widely adopted by other works, as evaluation metrics. For *WeChat* with implicit feedback, we use *Precision@k* (short as P@k) and *AUC*, two universally acknowledged metrics for 0-1 classification, to evaluate the performance. We repeated each experiment ten times and report the average performance.

Table 2: Comparative results for Epinions and WeChat. For MAE, RMSE, the smaller value is better, and vice versa for P@10, AUC.

	Epinions		WeChat	
	MAE	RMSE	P@10	AUC
SVD++ [15]	0.8321	1.0772	0.0653	0.7304
DELF [2]	0.8115	1.0561	<u>0.0752</u>	<u>0.7818</u>
TrustPro [37]	0.9130	1.1124	0.0561	0.6482
TrustMF [36]	0.8214	1.0715	0.0625	0.7005
TrustSVD [10]	0.8144	1.0492	0.0664	0.7325
NSCR [31]	0.8044	1.0425	0.0736	0.7727
SREPS [16]	<u>0.8014</u>	<u>1.0393</u>	0.0725	0.7745
DANSER	**0.7781**	**1.0268**	**0.0823**	**0.8165**
Impv.[1]	2.87%	1.25%	9.33%	4.48%

[1] The improvement compares DANSER with the best competitor (underlined).

4.2 Comparative Results: RQ1

We report experiment results of DANSER and other comparative methods in Table 2. As we can see, our model DANSER outperforms other comparative methods and achieves great improvements, i.e., 2.87% MAE impv. for Epinions and 4.48% AUC impv. for WeChat. The results verify the superiority of proposed model for social recommendation. Also, there are some findings in these comparative experiments. First, matrix factorization methods like TrustMF and TrustSVD can achieve good accuracy in Epinions with explicit feedback, but perform poorly in WeChat with implicit feedback. Second, NSCR performs quite better than other linear methods, which indicates that neural network plays an important role in capturing complex non-linear information. Third, even though DELF has not leveraged any social information, it outperforms some social recommendation methods like TrustSVD and NSCR. This is possibly due to the facts: i) the dual embedding and neural interaction layer in DELF can better exploit information in user-item interactions; ii) other social recommendation methods like TrustSVD and NSCR do not make good use of social information. Indeed, TrustSVD assumes equal importance of each friend user while NSCR uses graph regularization that constrains representations for adjacent users in a static way. These settings resort to a prior assumption that friend users' preferences are all similar, which may not be true for contemporary SNS (also discussed in Section 1).

4.3 Ablation Study: RQ2

In order to verify the effectiveness of some components in our model, we do some ablation studies and the results are shown in Table 3. We first compare DANSER with DualEMB and DualGCN. As we can see, GCNs can capture enough social information hidden

in graph structure and improve recommendation accuracy (about 1.0% impv. for MAE over DualEMB), while GATs can further improve performance by differentiate importance weights for friend users (about extra 0.8% impv. for MAE over DualGCN). We secondly compare DANSER with userGAT and itemGAT. The results show that incorporation of user and item networks can both bring significant performance improvement, and the benefits brought by user social network are greater.

We also evaluate the necessity of policy-based fusion layer. The results show that our policy-based fusion and policy gradient method can improve the accuracy a lot. Also, for Epinions, the improvement is quite significant. One possible reason is that the social effects in Epinions are more heterogeneous than those in WeChat, so to dynamically weigh different social effects are more effective for Epinions. More importantly, since DualEMB can be seen as the extension of DELF [2], where we replace the concatenation of four embedding by our policy unit. Comparing DualEMB in Table 3 with DELF in Table 2, we find that the policy unit do help to improve its performance and is superior to straightforward concatenation.

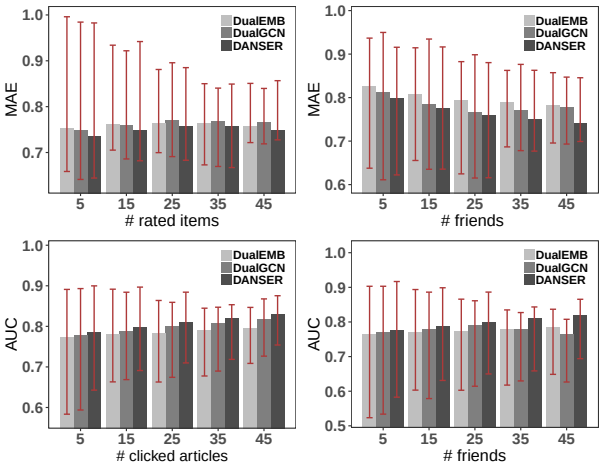

Figure 4: MAE/AUC of DANSER, DualGCN, DualEMB on Epinions/WeChat w.r.t users with different number of clicked items and friends.

In Section 1, we emphasized that social recommendation is proposed to solve the data sparsity and cold-start problem in recommender systems. Here we would like to investigate the performance of DANSER for cold-start users. Fig. 4 shows the results on Epinions and WeChat for users with different number of clicked items and friends. We present the median, the 75− and 25−percentile MAE/AUC. As we can see, with number of clicked items increasing, the gap between upper and lower bounds becomes narrow, which indicates that performance becomes better. Generally, DANSER can achieve better overall performance as well as lower variation of MAE/AUC. For users with different friend numbers, as depicted in the figure, when friend number goes up, the variation trends for three methods are different. For DualEMB, the performance improves a little. For DualGCN, there is an obvious performance decline, while performance of DANSER becomes better. This is possibly because for users with large number of friends, the GCN operation may incorporate more misleading information, and instead, GAT can attentively filter the noise and concentrate more on important neighbor nodes.

Table 3: Ablation study of components in proposed method.

	Epinions		WeChat	
	MAE	RMSE	P@10	AUC
DualEMB	.7920(1.7%)[1]	1.0363(0.7%)	.0794(3.6%)	.7992(2.2%)
DualGCN	.7840(0.7%)	1.0335(0.4%)	.0814(1.1%)	.8102(0.8%)
userGAT	.7858(0.9%)	1.0364(0.7%)	.0813(1.2%)	.8136(0.4%)
itemGAT	.7919(1.7%)	1.0335(0.4%)	.0813 (1.2%)	.8138(0.3%)
DANSER-w	.8191(4.9%)	1.0659(3.4%)	.0820(0.4%)	.8151(0.2%)
DANSER-m	.8211(5.2%)	1.0681(3.6%)	.0815(1.0%)	.8144(0.3%)
DANSER-a	.8232(5.4%)	1.0710(3.9%)	.0814 (1.1%)	.8140(0.3%)
DANSER-c	.8091(3.8%)	1.0659(3.4%)	.0809(1.7%)	.8118(0.6%)
DANSER	**0.7787**	**1.0292**	**0.0823**	**0.8165**

[1] The ratio indicates the impv. comparing DANSER with corresponding variant.

4.4 Parameter Sensitivity: RQ3

We study the performance variation for our model w.r.t some hyper-parameters including regularization parameter λ, dropout rate ρ, embedding dimension D, sample size F. (The other hyper-parameters have little impact on model performance, so we skip discussions on them for space limit.) The results are shown in Fig. 5 and enlighten us that. i) The regularization parameter plays an important role, and if it is too large, the model cannot focus on minimizing the recommendation loss. ii) If the dropout ratio is too large, the random pattern would hinder the training. If ρ is too small, it would lose the regularization efficacy. iii) A proper embedding size is needed. If it is too small, the model lacks expressiveness, while if it is too large, the representation vector would become so sparse, which leads to performance decline. iv) With the sample size increasing, the accuracy improves but the computational cost increases as well. Hence an appropriate sample size can keep a good balance between complexity and accuracy.

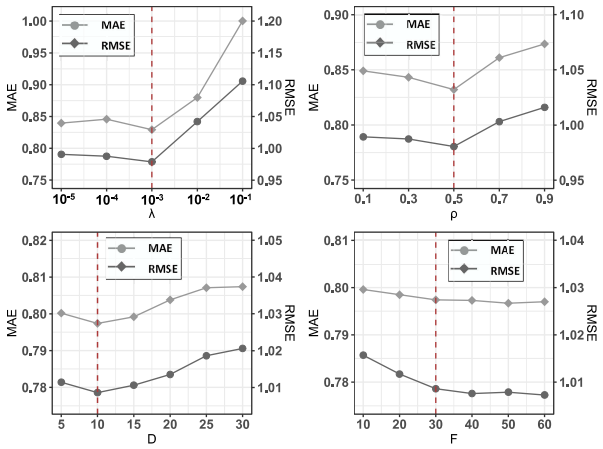

Figure 5: MAE of DANSER on Epinions w.r.t different hyper-parameters. The vertical dotted lines mark the values set in this paper.

4.5 Case Study: RQ4

We conduct a case study of four user-item pairs and show the weights given by four GATs by node size in Fig. 6. As we can see, the weights of two homophily GATs stay unchanged for the same user or item, while the weights of two influence GATs vary for different user-item pairs. Moreover, influence GATs incline to allocate larger weights to nodes with the same labels, which echoes the argument in Section 3.3 that the influence GATs tend to concentrate more on the similar nodes under a specific context. Also, the policy weights play a part in selecting the most dominant interacted features from four GATs representations.

More importantly, we can decompose the model interpretability for social effects in recommender systems by a two-level illustration: i) for a specific user-item pair (u, i), the policy weights first indicate the dominant combinations of social effects in user domain and item domain, ii) the corresponding GATs for four social effects further give out the influential degree of each friend given item i as well as the influential degree of each related item given user u.

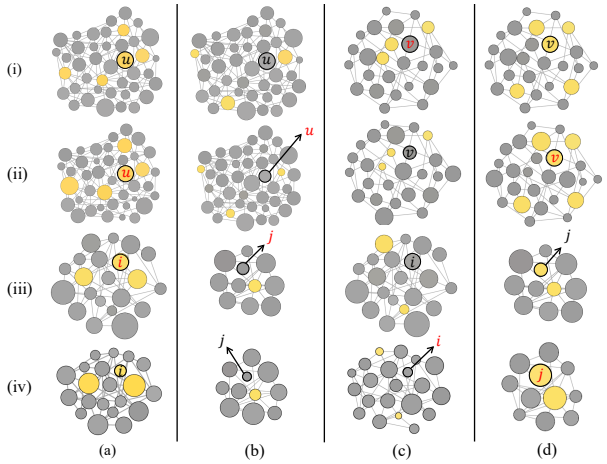

Figure 6: Visualization of the four GATs' weights for a case study. There are four user-item pairs: (a) (u, i), (b) (u, j), (c) (v, i), (d) (v, j). Row (i) and row (ii) are for social homophily and social influence GATs' weights, respectively, while row (iii) and row (vi) are for item-to-item homophily and item-to-item influence GATs' weights, respectively. Gray nodes are negative samples ($r_{ui} = 0$) while orange nodes are positive ones ($r_{ui} = 1$). In each line, the interacted feature of two factors with red symbol corresponds to the largest active probability given by policy networks.

5 RELATED WORKS

Social Recommendation. In online social networks, friends tend to have a large impact on users' decisions and behaviors [9, 26]. There are plenty of studies for social effects, such as homophily[23] and influence[20], over graph structures in social network analysis. For recommender systems, [13] is one of the pioneer who takes advantage of social networks to alleviate the data sparsity problem for item recommendation. Then many following studies begin to model social effects in recommender systems from various perspectives. [5, 12, 21] propose trust propagation methods by leveraging rating of friends to deduce rating of targeted user. This kind of approach is quite heuristic and is called *similarity-based* method. Then a more modern approach, *model-based* method, is proposed by other studies. [19, 38] use regularization to constrain the neighbor users' embedding vectors to be similar and [8, 17, 36] adopt matrix factorization to incorporate the adjacent matrix into user embedding. Since the above methods can only capture linear information, some recent studies like [4, 31] and [16, 32, 41] leverage deep neural networks and network embedding approaches, respectively, to learn a more complex representation for graph structures. However, comparing with dual GATs used in this paper, the common limitations of existing studies lie in: i) they assume neighbors' influences to be equally important or statically constrained, ii) they ignore the social effects from related items, iii) modeling of social effects lacks interpretability.

Graph Convolution/Attention Network. GCN and GAT, as two powerful techniques to encode a complex graph into low-dimensional representations, are extensively applied into various problems involved with graph data since their proposals [3, 14, 30]. Using GCN and GAT to solve semi-supervised classification problem in graph could achieve state-the-of-art performance (GAT can be seen as an extension of GCN and provide better performance according to [30]), and its good scalability enables it to tackle large-scale dataset. Existing studies leverage GAT to tackle social influence analysis[24], graph node classification[6], conversation generation[44], relevence matching[41]. For recommendation, some recent studies like [39] adopt GCN to convolve on user-item network (a bipartite graph) to obtain better representations for items, and there are also some works using GCN to capture social information for recommendation [40][34]. We are the first to use GAT for social recommendation task, and our new architecture, dual GATs, can capture social information in both user and item networks.

Dual Mechanism. There are many dual phenomenons in real-life which inspire several dual structures in model design. [35] proposes a dual-model paradigm to merge the training of two dual tasks. [45] extends GCN to dual structures, combining global and local consistencies in graph. [2] proposes DELF conducting dual embedding for users and items in recommender systems. Compared to [2], our model possesses several key differences: i) for the item-based user embedding (resp. the user-based item embedding), DANSER considers dynamic embedding under specific candidate items (resp. targeted users), while DELF treats embedding statically given a user (resp. item); ii) in dual GCN/GAT layer, DANSER harnesses two dual GATs to capture social information in user social network and item implicit network, while DELF considers no user-to-user and item-to-item mutual influences; iii) in the policy-based fusion layer, DANSER dynamically weighs the importances of the four GAT representations, while DELF straightforwardly concatenates four embedding vectors.

6 CONCLUSION

In this paper, we proposed DANSER, which includes two dual graph attention networks, to learn deep representations for social effects in recommender systems. Our method is equipped with good expressiveness because: i) it can collaboratively model homophily

and influence effects from both friend users and related items; ii) it can investigate heterogeneous importance distributions among different interactions of the two two-fold social effects for distinct user-item pairs. Our comparative experiments and ablation studies on a benchmark dataset and a commercial dataset showed that DANSER can learn effective representations of multifaceted social effects and leverage this information to significantly improve recommendation accuracy.

As extension of our work, we believe that the construction of item implicit network could leverage extra attribute features or distill more high-level connections through knowledge graphs if the information is available. More generally, one could extend DANSER to other information retrieval tasks, like question answering, where DANSER could model 'social effects' from related questions and similar answers given the corresponding networks of them.

REFERENCES

[1] Sébastien Bubeck and Nicolò Cesa-Bianchi. 2012. Regret Analysis of Stochastic and Nonstochastic Multi-armed Bandit Problems. *Foundations and Trends in Machine Learning* 5, 1 (2012), 1–122.

[2] Weiyu Cheng, Yanyan Shen, Yanmin Zhu, and Linpeng Huang. 2018. DELF: A Dual-Embedding based Deep Latent Factor Model for Recommendation. In *IJCAI*. 3329–3335.

[3] Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. In *NIPS*. 3837–3845.

[4] Wenqi Fan, Qing Li, and Min Cheng. 2018. Deep Modeling of Social Relations for Recommendation. In *AAAI*. 8075–8076.

[5] Jennifer Golbeck. 2006. Generating Predictive Movie Recommendations from Trust in Social Networks. In *iTrust*. 93–104.

[6] Liyu Gong and Qiang Cheng. 2018. Adaptive Edge Features Guided Graph Attention Networks. *CoRR* abs/1809.02709 (2018).

[7] Amit Goyal, Francesco Bonchi, and Laks V. S. Lakshmanan. 2010. Learning influence probabilities in social networks. In *WSDM*. 241–250.

[8] Yulong Gu, Jiaxing Song, Weidong Liu, Lixin Zou, and Yuan Yao. 2018. CAMF: Context Aware Matrix Factorization for Social Recommendation. *Web Intelligence* 16, 1 (2018), 53–71.

[9] Adrien Guille and Hakim Hacid. 2012. A predictive model for the temporal dynamics of information diffusion in online social networks. In *WWW*. 1145–1152.

[10] Guibing Guo, Jie Zhang, and Neil Yorke-Smith. 2015. TrustSVD: Collaborative Filtering with Both the Explicit and Implicit Influence of User Trust and of Item Ratings. In *AAAI*. 123–129.

[11] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *WWW*. 173–182.

[12] Mohsen Jamali and Martin Ester. 2009. TrustWalker: a Random Walk Model for Combining Trust-based and Item-based Recommendation. In *KDD*. 397–406.

[13] Henry A. Kautz, Bart Selman, and Mehul A. Shah. 1997. Referral Web: Combining Social Networks and Collaborative Filtering. *Communication of ACM* 40, 3 (1997), 63–65.

[14] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*.

[15] Yehuda Koren. 2008. Factorization Meets the Neighborhood: a Multifaceted Collaborative Filtering Model. In *KDD*. 426–434.

[16] Chun-Yi Liu, Chuan Zhou, Jia Wu, Yue Hu, and Li Guo. 2018. Social Recommendation with an Essential Preference Space. In *AAAI*. 346–353.

[17] Hao Ma, Irwin King, and Michael R. Lyu. 2009. Learning to Recommend with Social Trust Ensemble. In *SIGIR*. 203–210.

[18] Hao Ma, Haixuan Yang, Michael R. Lyu, and Irwin King. 2008. SoRec: Social Recommendation using Probabilistic Matrix Factorization. In *CIKM*. 931–940.

[19] Hao Ma, Dengyong Zhou, Chao Liu, Michael R. Lyu, and Irwin King. 2011. Recommender Systems with Social Regularization in Social Networks. In *WSDM*. 287–296.

[20] Peter V Marsden and Noah E Friedkin. 1994. Network studies of social influence. *Sociological Methods Research* 22, 1 (1994), 127–151.

[21] Paolo Massa and Paolo Avesani. 2007. Trust-Aware Recommender Systems. In *KDD*. 17–24.

[22] Paolo Massa and Paolo Avesani. 2007. Trust-aware recommender systems. In *RecSys*. 17–24.

[23] Miller Mcpherson, Lynn Smith-Lovin, and James M. Cook. 2001. Birds of a Feather: Homophily in Social Networks. *Annual Review of Sociology* 27, 1 (2001), 415–444.

[24] Jiezhong Qiu, Jian Tang, Hao Ma, Yuxiao Dong, Kuansan Wang, and Jie Tang. 2018. DeepInf: Social Influence Prediction with Deep Learning. In *KDD*. 2110–2119.

[25] Badrul Munir Sarwar, George Karypis, Joseph A. Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *WWW*. 285–295.

[26] Hossam Sharara, William Rand, and Lise Getoor. 2011. Differential Adaptive Diffusion: Understanding Diversity and Learning whom to Trust in Viral Marketing. In *ICWSM*.

[27] Jiliang Tang, Xia Hu, and Huan Liu. 2013. Social recommendation: a review. *Social Netw. Analys. Mining* 3, 4 (2013), 1113–1133.

[28] Hanghang Tong, Christos Faloutsos, and Jia-Yu Pan. 2006. Fast Random Walk with Restart and Its Applications. In *ICDM*. 613–622.

[29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *NIPS*. 6000–6010.

[30] Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *ICLR*.

[31] Xiang Wang, Xiangnan He, Liqiang Nie, and Tat-Seng Chua. 2017. Item Silk Road: Recommending Items from Information Domains to Social Users. In *SIGIR*. 185–194.

[32] Yufei Wen, Lei Guo, Zhumin Chen, and Jun Ma. 2018. Network Embedding Based Recommendation Method in Social Networks. In *WWW*. 11–12.

[33] Ronald J. Williams. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning* 8 (1992), 229–256.

[34] Le Wu, Peijie Sun, Richang Hong, Yanjie Fu, Xiting Wang, and Meng Wang. 2018. SocialGCN: An Efficient Graph Convolutional Network based Model for Social Recommendation. *CoRR* abs/1811.02815 (2018).

[35] Yingce Xia, Xu Tan, Fei Tian, Tao Qin, Nenghai Yu, and Tie-Yan Liu. 2018. Model-Level Dual Learning. In *ICML*. 5379–5388.

[36] Bo Yang, Yu Lei, Dayou Liu, and Jiming Liu. 2013. Social Collaborative Filtering by Trust. In *IJCAI*. 2747–2753.

[37] Mao Ye, Peifeng Yin, Wang-Chien Lee, and Dik Lun Lee. 2011. Exploiting geographical influence for collaborative point-of-interest recommendation. In *SIGIR*. 325–334.

[38] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2010. A Matrix Factorization Technique with Trust Propagation for Recommendation in Social Networks. In *RecSys*. 135–142.

[39] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *KDD*. 974–983.

[40] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *KDD*. 974–983.

[41] Ting Zhang, Bang Liu, Di Niu, Kunfeng Lai, and Yu Xu. 2018. Multiresolution Graph Attention Networks for Relevance Matching. In *CIKM*. 933–942.

[42] Xingang Zhang, Qijie Gao, Christopher S.G. Khoo, and Amos Wu. 2013. Categories of Friends on Social Networking Sites: An Exploratory Study. In *A-LIEP*. 244–259.

[43] Guorui Zhou, Xiaoqiang Zhu, Chengru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *KDD*. 1059–1068.

[44] Hao Zhou, Tom Young, Minlie Huang, Haizhou Zhao, Jingfang Xu, and Xiaoyan Zhu. 2018. Commonsense Knowledge Aware Conversation Generation with Graph Attention. In *IJCAI*. 4623–4629.

[45] Chenyi Zhuang and Qiang Ma. 2018. Dual Graph Convolutional Networks for Graph-Based Semi-Supervised Classification. In *WWW*. 499–508.

Adversarial Training Model Unifying Feature Driven and Point Process Perspectives for Event Popularity Prediction

Qitian Wu
Shanghai Jiao Tong University
echo740@sjtu.edu.cn

Chaoqi Yang
Shanghai Jiao Tong University
ycqsjtu@gmail.com

Hengrui Zhang
Shanghai Jiao Tong University
sqstardust@sjtu.edu.cn

Xiaofeng Gao*
Shanghai Jiao Tong University
gao-xf@cs.sjtu.edu.cn

Paul Weng
Shanghai Jiao Tong University
paul.weng@sjtu.edu.cn

Guihai Chen
Shanghai Jiao Tong University
gchen@cs.sjtu.edu.cn

ABSTRACT

This paper targets a general popularity prediction problem for event sequence, which has recently gained great attention due to its extensive applications in various domains. Feature driven method and point process method are two basic thinking paradigms to tackle the prediction problem, but both of them suffer from limitations. In this paper, we propose PreNets unifying the two thinking paradigms in an adversarial manner. On one side, feature driven model acts like a 'critic' who aims to discriminate the predicted popularity from the real one based on a set of temporal features from the sequence. On the other side, point process model acts like an 'interpreter' who recognizes the dynamic patterns in sequence to generate a predicted popularity that can fool the 'critic'. Through a Wasserstein learning based two-player game, the training loss of the 'critic' guides the 'interpreter' to better exploit the sequence patterns and enhance prediction, while the 'interpreter' pushes the 'critic' to select effective early features that helps discrimination. This mechanism enables the framework to absorb the advantages of both feature driven and point process methods. Empirical results show that PreNets achieves significant MAPE improvement for both Twitter cascade and Amazon review prediction.

KEYWORDS

Event Sequence, Popularity Prediction, Adversarial Model, Feature Driven, Point Process

ACM Reference Format:
Qitian Wu, Chaoqi Yang, Hengrui Zhang, Xiaofeng Gao, Paul Weng, and Guihai Chen. 2018. Adversarial Training Model Unifying Feature Driven and Point Process Perspectives for Event Popularity Prediction. In *The 27th ACM International Conference on Information and Knowledge Management (CIKM '18), October 22–26, 2018, Torino, Italy*. ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3269206.3271714

*Xiaofeng Gao is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CIKM '18, October 22–26, 2018, Torino, Italy
© 2018 Association for Computing Machinery.
ACM ISBN 978-1-4503-6014-2/18/10…$15.00
https://doi.org/10.1145/3269206.3271714

1 INTRODUCTION

Event sequences are ubiquitous in various domains and can model many real-life phenomenons, such as retweet cascade of one Twitter post, purchase behaviors of one Amazon product, and infection of one epidemic disease. For Twitter post, event sequences record the occurring time of each retweet. For Amazon product, event sequences record the time each purchase or click behavior happens. For epidemic disease, event sequences consist of the time each infection takes place. Given an observed event sequence with limited length, one would usually like to know how many events would occur in the end, i.e., the *event popularity*. In social networks, predicting the popularity of an original tweet can help conduct rumor monitoring [38] and anomaly detection [6]. In e-commerce, predicting the popularity of a new product can be used for personalized recommendation [19] and targeted advertisement [15]. In the health-care domain, if one can predict the influence of an epidemic disease, then preventive measures could be taken to control the infection evolution. Building an effective model to predict event popularity can bring many economic and social benefits. Fig. 1 shows a flowchart which compares the two mainstream thinking paradigms that are used to tackle the prediction problem.

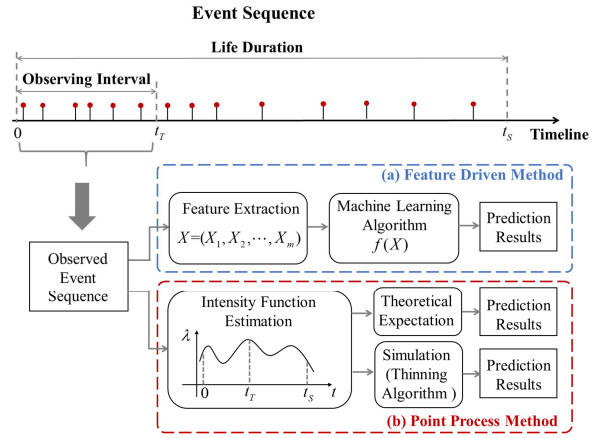

Figure 1: Two methods for event popularity prediction.

One thinking paradigm from a macroscopic view is feature driven method (Fig. 1(a)). As the name implies, this method first extracts a set of early features from observed event sequence and then leverages some machine learning algorithms to determine a mapping function from early features to popularity. Various features from different perspectives have been proved to be good early

popularity indicators, such as user profiles [24], user history activities [41], social network properties [10], and temporal characteristics of observed sequences [10, 16, 31]. While feature driven method can achieve decent accuracy and possess good robustness, it suffers from two limitations: i) *high human expertise*: feature selection requires a deep knowledge of a domain and lots of human intervention; ii) *high feature cost*: some models leverage user profiles or social network information, which could be inaccessible in practical scenarios due to some privacy issues.

Another thinking paradigm, point process method, uses a more principled approach (Fig. 1(b)). This method views an event sequence as a counting process and models the time intervals between events as random variables. One way to characterize the point process is by the conditional intensity function designed to model the time of next event occurrence given the history event sequence. Different forms of intensity functions can capture different dynamic patterns in the sequences, like inhomogeneous Poisson Process [30], Weibull model [42], and Hawkes Process [4, 18, 27, 43]. Based on the observed sequence, the intensity function can be estimated via Maximum Likelihood Estimation (MLE). After that, there are two ways to predict the popularity (in Fig. 1(b)). One is to derive the theoretical expectation of the event number when time goes to infinity [22, 43]. The other is to simulate the occurrence of future events by the thinning algorithm [9, 23]. While the point process method is equipped with solid theoretical foundations [7], it also has some limitations: i) *long observation dependency*: the precision highly relies on sufficient observation, which disqualifies it to accommodate early-stage prediction with short observed sequences; ii) *model failure*: the expectation of event number may be infinite in a supercritical region [43], which makes the model output an invalid predicted value.

Due to the limitations of feature driven and point process methods, solely adopting one method is not a satisfying solution, especially when the following two situations are encountered: i) early-stage prediction is needed, and ii) other features from social networks or user profiles are inaccessible. In this paper, inspired by Generative Adversarial Nets (GAN) [12] and its Wasserstein variant called WGAN [3], we propose PreNets (Adversarial Nets for Event Popularity Prediction) to combine the feature driven and point process methods through an adversarial training rule. On one side, point process model plays the role of what we call an 'interpreter', recognizing the patterns hidden in observed event sequence to generate the predicted popularity. On the other side, feature driven model plays as a 'critic', firstly extracting a set of temporal features from observed sequences and then using them to discriminate the predicted popularity given by the 'interpreter' from the real one. The Earth Moving (EM) distance, also known as Wasserstein distance [3], is adopted to measure the discrepancy between the predicted value and the ground-truth label.

The training stage is a two-player minimax game. The 'critic' aims at maximizing the EM distance based on the features, and then guides the training of the 'interpreter'. The 'interpreter' struggles to minimize the EM distance by giving a convincing predicted value, and then pushes the 'critic' to the limits. As the 'critic' becomes stronger, the 'interpreter' can obtain higher-quality guidance. This mechanism enables PreNets to take advantage of both thinking paradigms: i) point process model exploits the patterns hidden in sequence; ii) feature driven model leverages good early features to suit the model to early-stage prediction.

We apply PreNets to two real-world prediction tasks: retweet cascade on Twitter and movie review on Amazon. To validate the advantages of adversarial training, we show that PreNets could achieve better precision than independently trained point process and feature driven models. For comparison, we let PreNets compete with state-of-the-art models, and the results demonstrate the superiority of PreNets. Concretely, for Twitter cascade/Amazon review with short and long observation, PreNets respectively achieves 5.6%/7.2% and 21.1%/6.3% improvement on MAPE.

2 PRELIMINARY

Event sequence triggered by one source consists of a series of timestamps that record each event occurrence. For example, an attractive tweet on Twitter may cause a long retweet cascade, a popular product on Amazon could trigger streaming purchase or click behaviors, one epidemic disease would bring about an infection outbreak. Here, the tweet, the product, and the disease are called sources, while the retweet cascade, the user behaviors, and the infection process would form event sequences. For each case, one event happens when: i) a user forwards the tweet, ii) a user purchases the product or clicks on the webpage, iii) a new patient is infected by the disease. Here is a formal definition.

Definition 1. *(Event Sequence): For source i, we define the time sequence $\mathcal{S}^i(t) = \{t_j^i\}$ as event sequence up to time t, $1 \le j \le N^i(t)$. Here $\{t_j^i\}$ is the timestamp when event j happens and $N^i(t) = |\mathcal{S}^i(t)|$ denotes the event number for source i up to time t.*

Definition 2. *(Popularity): For source i, we define its popularity as $N^i(\infty) = \lim_{t\to\infty} |\mathcal{S}^i(t)|$. There exists t_s^i for source i such that $N^i(t_s^i) \approx N^i(\infty)$, and we call $[0, t_s^i]$ the life duration of source i.*

Point Process Theory. In point process theory, the time intervals between events are modeled as random variables. One point process can be characterized via conditional intensity function:

Definition 3. *(Conditional intensity function): For source i, $\lambda^*(t)$ is the expected instantaneous rate of new event occurrence given the history sequence:*

$$\lambda^*(t) = \lambda(t|\mathcal{S}^i(t)) = \frac{\mathbb{P}\{N^i(t+dt) - N^i(t) = 1|\mathcal{S}^i(t)\}}{dt}. \quad (1)$$

Here the notation $*$ serves as a reminder that the intensity is conditional on the history sequence.

In a small time interval $[t, t+dt)$, $\lambda^*(t)dt$ is the probability for a new event occurrence given the history sequence $\mathcal{S}^i(t)$. Then the conditional density function $f^*(t)$ for the time of next event since time t_n can be specified by[1]

$$f^*(t) = \lambda^*(t)\exp(-\int_{t_n}^{t} \lambda^*(\tau)d\tau). \quad (2)$$

Some particular forms of intensity functions are designed to capture different dynamic patterns in the sequence, such as inhomogenous Poisson Process [30], Weibull model [42], Hawkes Process with exponential kernel [4], power-low kernel [43], or Fourier based kernel [18].

[1]Please refer to [7] or a recent tutorial [26] for derivation.

Problem Formulation. The problem in this paper can be formulated as: for a new source i, given the observed event sequence $\mathcal{S}^i(t_T)$ (we call $[0, t_T]$ as the observing interval), we aim to predict its popularity $N^i(\infty)$. This problem is non-trivial due to the following two key points. 1) *Early-Stage Prediction*: The observing interval can be significantly short compared with the life duration of one event sequence. 2) *Poor Information Driven*: The available information is only limited to the source description and the observed sequence. Other features from user profiles, user history activities, social networks and some cross-platform information are unknown.

In the following parts, for compactness, we use $\mathcal{S}^i$, N^i, and n^i to simplify $\mathcal{S}^i(t_T)$, $N^i(\infty)$, and $N^i(t_T)$ respectively.

3 METHODOLOGY

In this section, we present our model PreNets that elegantly tackles the non-trivial prediction problem. Firstly, we cast a glance at an overview of the framework. Then we put forward the point process model and the feature driven model, which respectively plays as an 'interpreter' and a 'critic' in PreNets. Afterwards, we zoom in on the adversarial training for unifying two basic models by a minimax game.

3.1 Model Overview

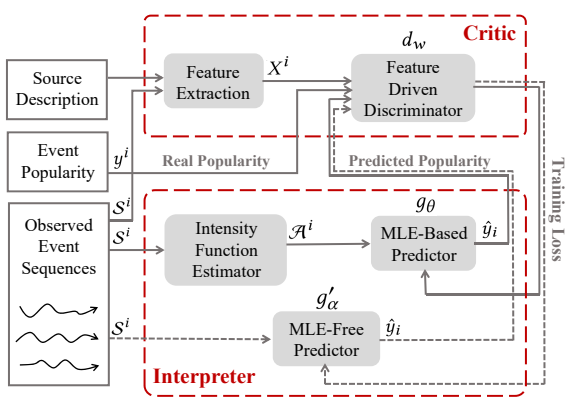

Figure 2: PreNets framework.

Fig. 2 is the framework of PreNets consisting of two parts: point process model ('interpreter') and feature driven model ('critic'). The inputs of PreNets are event sequence and its source description. Then the trained PreNets will output the predicted popularity for each event sequence.

The training procedure of PreNets is based on a minimax game between the 'interpreter' and the 'critic'. The 'interpreter' aims at using the observed sequence to predict its popularity. We propose two self-contained models to accomplish this mission from different perspectives. One is MLE-based point process model and the other is MLE-free point process model. For MLE-based model, we first use each event sequence to estimate an intensity functions (implemented by RNN) via maximum likelihood estimation (MLE). The estimated intensity functions can capture the dynamic patterns in the sequential data, and will be further input into a MLE-based predictor (implemented by another RNN). The predictor will output the predicted popularity. Differently, the MLE-free model skips the intensity function estimator, directly inputs the event sequences into the MLE-free predictor (implemented by RNN) and conducts prediction.

On the other side, feature driven model first extracts a set of features from each event sequence and basic information about the source (e.g., the posting time of one original tweet, the category of one product or the description of one epidemic disease). Then these features, the real popularity of each sequence as well as the predicted popularity given by the predictor will be input into a feature driven discriminator (implemented by a 3-layer ELU neural network), which is designed to distinguish the predicted value from the real one. Once we update the feature driven discriminator, the loss generated from the predicted popularity will be transferred back to the 'interpreter' as its feedback training signal. Afterwards, the predictor will be updated based on this feedback signal.

During the training stage, the 'interpreter' struggles to generate the popularity which can fool the 'critic', while the 'critic' tries its best to conduct the discrimination and then instructs the 'interpreter'. The game will be over once an equilibrium is reached. Then, the 'critic' can no longer distinguish the predicted popularity, and the 'interpreter' becomes a strong predictor. The framework absorbs the strengths of both feature driven and point process methods.

3.2 Interpreter: Point Process Model

The point process model aims at leveraging the observed sequence to predict its popularity. We propose two self-contained models to accomplish the prediction from different perspectives. In the following subsections, we will present our model design, its rationale, and the corresponding motivations.

3.2.1 MLE-based Point Process Model. As introduced in Section 2, one way to characterize the point process is via the conditional intensity function. A slew of prior works assume specific parametric forms for the intensity function, such as inhomogeneous Poisson Process or Hawkes Process. Some of them are claimed to be effective in some specific tasks. For instance, Hawkes Process with power-low kernel [22, 43] works pretty well when predicting the information cascade in social networks. While the parametric assumption guarantees the accuracy for one kind of event sequence, it limits the model generality to other kinds of event sequence.

Recurrent Neural Network (RNN) is a feedforward neural network architecture where the current state depends on both the current input signal and the previous state. Such mechanism enables the network to memorize the influence of history signals. RNN has been shown to be powerful in modeling sequential data, such as handwriting recognition [13], and machine translation [33]. [8, 39, 40] leverage RNN to model the intensity functions of point process and achieve state-of-the-art performance.

Following the frontier researches [8, 40], we relax the parametric assumption for the intensity functions and leverage RNN to approximate the conditional intensity for event sequence.

Intensity Function Estimation. Considering one point process for source i, given the observed sequence $\mathcal{S}^i = \{t_j^i\}$. Use $\boldsymbol{h}_j$ to denote the hidden state of RNN, and we update the hidden layer once receiving a new event,

$$\boldsymbol{h}_{j+1} = \max\left\{\boldsymbol{w}_h \cdot t_j^i + \boldsymbol{v}_h \cdot \boldsymbol{h}_j + \boldsymbol{b}_h, 0\right\}. \tag{3}$$

The conditional intensity function based on $\mathcal{S}^i(t_j^i)$ can be specified as

$$\lambda^*(t) = \exp(\boldsymbol{v}_t \cdot \boldsymbol{h}_j + w_t(t - t_j^i) + b_t), \tag{4}$$

where in the exponential function, the first term aggregates the influence of previous events, the second term captures the current influence by event j, and the last term stands for the base intensity. The outside exponential function acts as a non-linear transformation [8].

Then according to (2), the log-likelihood of a collection of sequences $C = \{\mathcal{S}^i\}$, where $\mathcal{S}^i = \{t_j^i\}_{j=1}^{n^i}$, can be derived as

$$\begin{aligned} l(C) &= \sum_i \sum_j \log f^*(t_{j+1}) \\ &= \sum_i \sum_j \mathbf{v}_t \cdot \mathbf{h}_m + w_t(t_{j+1}^i - t_j^i) + b_t + \frac{1}{w_t}\exp(\mathbf{v}_t \cdot \mathbf{h}_m + b_t) \\ &\quad - \frac{1}{w_t}\exp(\mathbf{v}_t \cdot \mathbf{h}_m + w_t(t_{j+1}^i - t_j^i) + b_t). \end{aligned} \tag{5}$$

By maximizing the log-likelihood, one can obtain the intensity functions $\lambda^*(t)$ for each source i. We use stochastic gradient descent as our optimization algorithm.

Popularity Prediction. We further probe into the prediction. Based on the intensity functions, we concatenate the intensity values with the original event timestamps and define an intensity sequence:

$$\mathcal{A}^i = \left\{A_j^i\right\}_{j=1}^{n^i}, \quad A_j^i = (t_j^i, \lambda^*(t_j^i)). \tag{6}$$

The intensity sequence records the timestamp of each event and the corresponding intensity value, which can reflect the dynamic patterns in one sequence. So one can consider $\mathcal{A}^i$ as a kind of representation for the observed event sequence.

Then we again adopt RNN to exploit the relationship between intensity sequence $\mathcal{A}^i$ and the popularity N^i, i.e., $g_\theta : \mathcal{A}^i \to N^i$. In our experiment, we also leverage more complex architecture, Long Short-Term Memory (LSTM) network, to implement g_θ, but here we use RNN to give a more concise illustration. Use $\boldsymbol{h}_j$ to denote the hidden state and $\hat{y}$ to denote the output result, we have

$$\boldsymbol{h}_{j+1} = \tanh(\boldsymbol{v}_h' \cdot A_j^i + \boldsymbol{w}_h' \cdot \boldsymbol{h}_j + \boldsymbol{b}_h'), \tag{7}$$

$$\hat{y} = \boldsymbol{w}_o \cdot \boldsymbol{h}_M + b_o, \tag{8}$$

where $M = \max n^i$ and the model parameter set $\theta = \{\boldsymbol{v}_h', \boldsymbol{w}_h', \boldsymbol{b}_h', \boldsymbol{w}_o, b_o\}$. Then the problem remains as how to train g_θ. We will focus on it in Section 3.4.

The MLE-based point process model first converts the event sequence to intensity sequence as a representation and then uses the intensity sequence to predict popularity. To obtain the intensity functions, one must optimize the log-likelihood function. Solving the MLE requires high computational complexity in practice, so we propose another point process model.

3.2.2 MLE-Free Point Process Model. Since our goal is to predict the popularity rather than to understand the patterns in the event sequence, we can skip the estimation of intensity functions and the MLE, directly targeting the relationship between observed sequence $\mathcal{S}^i$ and the popularity, i.e., $g_\alpha' : \mathcal{S}^i \to N^i$ by RNN. Concretely,

$$\boldsymbol{h}_{j+1} = \tanh(\boldsymbol{v}_h'' \cdot t_j^i + \boldsymbol{w}_h'' \cdot \boldsymbol{h}_j + \boldsymbol{b}_h''), \tag{9}$$

$$\hat{y} = \boldsymbol{w}_o' \cdot \boldsymbol{h}_M + b_o'. \tag{10}$$

The model parameter set $\alpha = \{\boldsymbol{v}_h'', \boldsymbol{w}_h'', \boldsymbol{b}_h'', \boldsymbol{w}_o', b_o'\}$.

To put it more symbolically, each point process model is like an 'interpreter', who works on interpreting the event sequence into a predicted popularity value. The MLE-based point process model first converts the sequence into an intensity sequence, and then conducts prediction, while the MLE-free model tackle the prediction directly. Experiments in Section 4 sheds more insights by comparison between these two models.

3.3 Critic: Feature Driven Model

Unlike point process model, feature driven method studies the prediction problem from the perspective of feature extraction. In our PreNets, different from previous works, we leverage feature driven model to discriminate the predicted popularity given by the 'interpreter' from the real one.

Feature Extraction. Extracted features should reflect the characteristics of observed event sequence from various aspects. The features used in this paper includes: number of events in the first/second half of observing interval, increment of events in the first/second half of observation interval, mean time interval between events in the first/second half of observing interval, maximum time interval between events, mean and standard deviation of event number per time unit, mean and standard deviation of time interval between events per time unit, five point summary of waiting time between events, five point summary of event number per time unit, coefficients of polynomial curve fitting for the cumulative event number. Note that all the features mentioned above can be easily obtained given the observed event sequence $\mathcal{S}^i$.

These features are proved to be effective early indicators of popularity in previous papers, and here we provide a brief justification for some of them. The volume of events in observing interval reflects the early influence of the source [16]. The time interval between events captures the early attractiveness of the source [31]. The increment of events characterizes the popular potentiality [16]. The maximum time interval is related to the possible dormant cycle [10]. The five point distribution statistics approximately describe the temporal similarity among different event sequences, while the fitting curve shows the basic evolving trending of event number.

Moreover, some descriptive information about the source can also be leveraged as features, such as the generation time of the source (e.g., the posting time of the original tweet) and the category of the source (e.g., the product category or the disease category). These source features may vary for different tasks.

Some related studies rely on other information, like user profiles, history records of user activities, and social network properties. While such features are claimed to be important early indicators of popularity, they tend to be inaccessible in practice due to privacy concerns. Hence, to make our model equipped with good

generality, we solely rely on source features and temporal features that can be easily extracted from observed event sequences. For a more thorough study, we will extend our model to such a case with more features in experiment section and study the performance improvement brought by more early features.

Loss Function. The Earth Moving (EM) distance [3], also called Wasserstein distance, measures the distance between two probability distributions. Our model adopts EM distance to quantify the discrepancy between the predicted popularity and the real one to enable the feature driven model to conduct discrimination.

For one source i, we define two vectors $\eta^j = (X^i, y^i)$ and $\zeta^i = (X^i, \hat{y}^i)$, where X^i contains the features for i, y^i is the real popularity value, and $\hat{y}^i$ is the predicted popularity given by point process model. Assume η obeys a probability distribution $\mathbb{P}_r$ and ζ obeys a probability distribution $\mathbb{P}_g$. The Earth Moving (EM) distance between two probability distributions can be defined as follows:

$$W(\mathbb{P}_r, \mathbb{P}_g) = \inf_{\phi \sim \Phi(\mathbb{P}_r, \mathbb{P}_g)} \mathbb{E}_{(\eta,\zeta)\sim\phi}[||\eta - \zeta||], \tag{11}$$

where $\Phi(\mathbb{P}_r, \mathbb{P}_g)$ denotes the set of all joint distributions $\phi(\mathbb{P}_r, \mathbb{P}_g)$ whose marginal are $\mathbb{P}_r$ and $\mathbb{P}_g$. (11) can be equivalently written as:

$$W(\mathbb{P}_r, \mathbb{P}_g) = \sup_{||d||_L \le 1} \mathbb{E}_{\eta\sim\mathbb{P}_r}[d(\eta)] - \mathbb{E}_{\zeta\sim\mathbb{P}_g}[d(\zeta)], \tag{12}$$

where the supremum is taken over all Lipschitz functions $d : \Omega \to \mathbb{R}$ and Ω denotes the value space for η and ζ.

(12) is hard to solve since we can not enumerate all Lipschitz functions. Instead, we assume d is parametrized with w and use a specific function form to approximate the search space on all Lipschitz functions. Then the problem boils down to solving

$$\max_{w, ||d_w||_L \le 1} \mathbb{E}_{\eta\sim\mathbb{P}_r}[d_w(\eta)] - \mathbb{E}_{\zeta\sim\mathbb{P}_g}[d_w(\zeta)]. \tag{13}$$

The more flexible form of d_w will give more accurate approximation, and we will leave its discussion in our experiments.

In fact, the output result of $d_w(\cdot)$ measures the confidence level of popularity value given the features (the higher $d_w(\cdot)$ means the more confident), and (13) measures the discrepancy between the confidence levels of predicted popularity and the real one. The maximum in (13) is to enlarge the distance between probability distributions of predicted popularity and the ground-truth label conditioned on extracted features, which contributes to a fine discrimination. So the feature driven model is like a 'critic', using features to judge if the predicted value is convincing. (Indeed, the 'critic' notion is used to describe discriminator in WGAN [3].)

3.4 Adversarial Training

In PreNets, we let the point process model and the feature driven model be trained in an adversarial way, based on Wasserstein learning proposed by [3]. The feature driven model aims at discriminating the predicted popularity given by the point process model from the real popularity, i.e., maximizing the EM distance (see (13)). The point process model tries to fool the feature driven model by giving a convincing predicted result, i.e., minimizing the EM distance. So the global training objective can be written as:

$$\begin{aligned}\min_{\theta} \max_{w, ||d_w||_L \le 1} & \mathbb{E}_{(X,y)\sim\mathbb{P}_r}[d_w(X, y)] \\ & - \mathbb{E}_{(X, g_\theta(\mathcal{A}))\sim\mathbb{P}_g}[d_w(X, g_\theta(\mathcal{A})],\end{aligned} \tag{14}$$

where $g_\theta(\mathcal{A}) = \hat{y}$ denotes the predicted result given by (7) and (8) of RNN[2]. (14) can be approximated by solving

$$\min_{\theta} \max_{w, ||d_w||_L \le 1} \frac{1}{B}\sum_{i=1}^{B}[d_w(X^i, y^i)] - \frac{1}{B}\sum_{i=1}^{B}[d_w(X^i, g_\theta(\mathcal{S}^i)], \tag{15}$$

where B is the batch size. Following [39], we further convert the Lipchitz constraints into the gradient regularization in the objective:

$$\begin{aligned}\min_{\theta} \max_{w} & \frac{1}{B}\sum_{i=1}^{B}[d_w(X^i, y^i)] - \frac{1}{B}\sum_{i=1}^{B}[d_w(X^i, g_\theta(\mathcal{S}^i)] \\ & - \gamma \sum_{i,j=1}^{B} \Big| \frac{|d_w(X^i, y^i) - d_w(X^j, g_\theta(\mathcal{S}^j)|}{|y^i - g_\theta(\mathcal{S}^j)|} - 1 \Big|,\end{aligned} \tag{16}$$

where γ is the weight for regularization.

Then we provide the training algorithm for PreNets in Alg. 1. Before conducting adversarial training, we pretrain d_w and g_θ. When pretraining g_θ, we minimize the mean square loss

$$\min \frac{1}{B}\sum_{i=1}^{B}(g_\theta(\mathcal{S}^i) - y^i)^2. \tag{17}$$

Then during the adversarial training stage, the feature driven model and the point process model are updated alternatively until convergence. Note that Alg. 1 is designed for MLE-based PreNets (MLE-based point process model v.s. feature driven model), and line 7, line 13 will be removed for MLE-free PreNets (MLE-free point process model v.s. feature driven model).

Algorithm 1: Minimax Game for PreNets

1 **REQUIRE:** γ, the weight for regularization. $n_{critic} = 6$, the number of iterations of the 'critic' per 'interpreter' iteration. $B = 64$, the batch size. $\alpha = 5e-4$, $\beta_1 = 0.9$, $\beta_2 = 0.9$, Adam hyper-parameters.
2 **REQUIRE:** w_0, initial 'critic' parameter. θ_0, initial 'interpreter' parameter.
3 Pretrain d_w and g_θ using data $C = \{\mathcal{S}^j\}$;
4 **while** *not converged* **do**
5 **for** $k = 0, \cdots, n_{critic}$ **do**
6 Sample $\{(X^i, y^i)\}_{i=1}^{B} \sim \mathbb{P}_r$ from real data;
7 Compute $\lambda^*(t)$ by (4) and get $\mathcal{A}^i$ by (6) for each i;
8 $L_C \leftarrow \frac{1}{B}\sum_{i=1}^{B}[d_w(X^i, g_\theta(\mathcal{S}^i)] - \frac{1}{B}\sum_{i=1}^{B}[d_w(X^i, y^i)]$
9 $+\gamma \sum_{i,j=1}^{B} \Big|\frac{|d_w(X^i, y^i) - d_w(X^j, g_\theta(\mathcal{S}^j)|}{|y^i - g_\theta(\mathcal{S}^j)|} - 1\Big|$;
10 $w \leftarrow$Adam(∇L_C, α, β_1, β_2);
11 **end**
12 Sample $\{(X^i, y^i)\}_{i=1}^{B} \sim \mathbb{P}_r$ from real data;
13 Compute $\lambda^*(t)$ by (4) and get $\mathcal{A}^i$ by (6) for each i;
14 $L_I \leftarrow \frac{1}{B}\sum_{i=1}^{B}[f_w(X^j, g_\theta(\mathcal{S}^i)]$;
15 $\theta \leftarrow$Adam($-\nabla L_I$, α, β_1, β_2);
16 **end**

The adversarial training is a two-player game between the 'interpreter' and the 'critic'. They are struggling to 'beat' each other, through which the potential energy of them can be exploited: i) the 'interpreter' could better recognize the patterns hidden in sequence; ii) the 'critic' could select effective early features that help discrimination. The game will terminate when an equilibrium is

[2]The derivation is designed for MLE-based point process model. For MLE-free model, one can replace the $g_\theta(\mathcal{A})$ by $g'_\alpha(\mathcal{S})$.

reached [12], where the 'critic' can no longer discriminate the predicted popularity and the 'interpreter' can provide prediction which synthesizes the information contained in sequence patterns and the temporal features. Hence, the framework absorbs the strengths of both point process and feature driven methods. Our experiments will validate the advantages of adversarial training over independently training g_θ and d_w.

3.5 Links to Related Work

In order to shed more lights on our methodology, we compare PreNets with some existing works from three-fold perspectives.

3.5.1 Popularity Prediction. Most early studies on event popularity prediction are based on feature driven method. [32], as one of the pioneer setting foot in predicting single tweet popularity, uses generalized linear model to find the relationship between some early features and the retweet count. In addition, a series of works have tried various features from user profiles [24], history activity records [41], social networks [10], and some cross-platform information [34]. Afterwards, point process models [4, 30, 43] are proposed to tackle the prediction problem from a more principled way. With good interpretability and solid theoretical foundations, point process models have been widely adopted by many researchers [9, 27, 29, 37]. However, these two methods suffer from respective limitations as discussed in Section 1.

Moreover, the early studies on popularity prediction rely heavily on long-term observation. Motivated by this, some recent researches target the early-stage prediction by probabilistic model [21] and regression tree [22]. However, they leverage some other information from social networks [21] or user profiles [22] to make up for the information poverty led by insufficient observation.

Compared with previous works, our PreNets owns the following advantages: i) *Model Unification*: PreNets unifies point process and feature driven models through a minimax game, absorbing the strengths of both two thinking paradigms. ii) *Generality*: Our point process model makes no parametric assumption for the intensity functions and our feature driven model only relies on features that can be directly extracted from the event sequences. These two facts enable our model to be generalized to any prediction task for sequential data, while most existing works target only one or some kinds of event sequence and make several task-specific assumptions or designs. iii) *Low Feature Cost*: Our model does not leverage other information such as user profiles or social network information, which are claimed to be indispensable in some other works. Our experiments show that PreNets is competent in early-stage prediction even with only temporal information.

3.5.2 Hybrid Prediction Model. Several recent studies make an attempt to incorporate feature driven and point process perspectives in one prediction model. [35] and [11] leverage survival theory to combine features with the point process model. The features are used to describe some characteristics of the events, and are input into the survival model together with the event timestamps. [22] proposes a hybrid model, in which the MLE-based Hawkes Process model is used to conduct a first-layer estimation of the popularity and then the features are applied to add an adjusting weight on the first-layer estimation to refine the precision. The results show that the hybrid model outperforms some state-of-the-art models for information cascade prediction. [20] proposes a lifetime aware regression method to predict the popularity of YouTube video. Some early features are used to determine the life spans of videos, based on which the videos would be divided into different groups. Videos in one group share the same parametric form of the regression function.

In contrast with them, our PreNets is quite different in the following four aspects. First, in PreNets, the feature driven model and the point process model are two self-contained models, instead of combined into a new model like [11, 35] or concatenated as two layers of one model [20, 22]. Second, we leverage adversarial training to unify two models and the objective is to search an equilibrium of the minimax game, instead of solving a MLE function or minimizing the mean square error. Third, in the adversarial training stage, the goal of the feature driven model is to discriminate the predicted value from the real one, so the features are used to help the 'critic' conduct discrimination rather than prediction. Also, whether [22] or [20] uses a lot of features from user profiles and social networks, which are not mandatory in PreNets.

3.5.3 Adversarial Training. Generative Adversarial Nets (GAN) has shown its great power in image and text generation as well as semi-supervised learning. The original GAN in [12] aims at minimizing the Jensen-Shannon divergence but is shown to suffer from highly instability and collapse mode [2, 28]. To refine the learning of GAN, [3] proposes Wasserstein GAN (WGAN) which adopts the Earth Moving (EM) distance as the objective for training. It is proved that using EM distance as the metric between probability distributions possesses many advantages, including reducing the mode dropping [14] and refining the generated samples. Our work extends Wasserstein learning to event popularity prediction problem but there are some differences. Firstly, the point process model aims at predicting the popularity from the observed sequence, instead of generating a complete event sequence based on the input of random data (like [39]) , which can be viewed as a counterpart for image generator using GAN. Hence, we call our point process model as an 'interpreter' throughout the paper, instead of a generator. The other difference lies in the feature driven model. In original GAN and WGAN, the generated pictures by the generative model would be the input of the discriminative model as the negative samples, while in PreNets, the features of one sequence are from the real sequence data, rather than the point process model.

There have been lots of successful applications of GAN in many domains, such as information retrieval [36], text to image synthesis [25], dialogue generation [17] and cryptography [1]. The proposal of GAN paves a way to unify two different models in an adversarial manner. Compared with other GANs unifying the generative and discriminative models, more precisely, PreNets unifies feature driven and point process models. In other words, PreNets is custom-made for event popularity prediction problem.

4 EXPERIMENTS

In this section, we apply our PreNets to two real-world prediction tasks : *retweet cascade* on Twitter and *movie review* on Amazon. The difference between two tasks lies in the different dynamic patterns of the event sequences. For retweet cascade, it is proved that

the previous retweet behaviors may have a strong correlation with the future retweet [5], since the users in social networks are prone to be influenced by the posts of whom they follow. For movie review, this correlation is not that significant. Another difference is that the life durations for most retweet cascades are one or two days, while the review sequence for one movie may span several months.

In the following parts, we will first introduce the experiment setup including the data set information, the evaluation protocol, and the comparative methods. Then we focus on the experiment results and do some discussions.

4.1 Experiment Setup

4.1.1 Date Sets. Our experiments are conducted on two large data sets.

Twitter. We use the public data provided by [43] to construct our Twitter data set. This data set contains roughly 160 thousand retweet cascades in total. For each cascade, it records the timestamps of each tweet post. We first filter the cascades with length between 50 and 5000. Then since our shortest observation time is five minutes, we drop the cascades whose number of tweets posted in the first five minutes are less than 10. Finally, we construct totally 21,890 cascades for our experiments.

Amazon. This data set is provided by Stanford Large Network Dataset Collection. The data ranges from Aug. 1997 to Oct. 2012, including approximately 8 million reviews, 250 thousand movies and 890 thousand users. Also, we first filter the movies with review number between 50 and 8000. Then we only consider the movies whose number of reviews in the first twenty days are more than 10. After that, we obtain 32,134 movies. This data set also provides user information and the plaintext of reviews. We basically ignore this extra information and will further discuss the improvement brought by incorporation of these features.

4.1.2 Evaluation Protocol. For each data set, we randomly choose 80% event sequences as training set and the remaining sequences as test set. We basically set two different observation times for comparison between prediction with short observation (early-stage prediction) and long observation. Concretely, for Twitter cascade, we consider 10 minutes and 1 hour observation. For Amazon review, we consider 20 days and 100 days observation. To study the impact of different observation time, we will probe into the performance variation as the observation time increases gradually.

We adopt different metrics to measure the prediction precision in a multifaceted way:

i) *Mean Absolute Percent Error (MAPE)* is the most commonly used metric to measure the precision for regression task. MAPE is defined as:

$$MAPE = \sum_i \frac{|\hat{y}^i - y^i|}{y^i}, \tag{18}$$

where $\hat{y}^i$ and y^i denote the predicted value and the ground-truth value, respectively.

ii) *Kendall Rank Coefficient* is defined as

$$Kendall = \frac{2(C(\hat{y}, y) - D(\hat{y}, y))}{n(n-1)}, \tag{19}$$

where $C(\hat{y}, y)$ and $D(\hat{y}, y)$ represent the number of concordant pairs and discordant pairs for $\hat{y}$ and y, respectively. This metric measures the rank similarity between predicted popularity values and the ground-truth labels.

iii) *Coverage@k* means the precision for the top k popular sources, i.e., the ratio of the sources with top k predicted popularity that are truly the top k popular sources. This metric counts the detection accuracy for some extreme popular items. We consider $k = 20$ and $k = 50$ in our experiments.

4.1.3 Comparative Methods. We select two state-of-the-art methods to compete with PreNets. One is the *Hybrid* model proposed by [22], which combines Hawkes process and feature driven perspectives. The other is the *LARM* [20], using features to separate different sequences and then adopting regression method to conduct prediction.

Moreover, in order to verify the advantages of adversarial training, we also independently train the feature driven model and the point process model to fit the data by minimizing mean square error

$$\frac{1}{B}\sum_{i=1}^{B}(h(\cdot) - y^i)^2. \tag{20}$$

The $h(\cdot)$ in (20) is implemented in distinct way for each model. For feature driven model, it is implemented by a 3-layer ELU neural network with feature vector X^i as input. For MLE-based point process model, it is implemented by LSTM with estimated intensity sequence $\mathcal{A}^i$ as input. For MLE-free point process model, it is implemented by LSTM with observed event sequence $\mathcal{S}^i$ as input.

In the following parts, we use *FD*, *PP-MB*, *PP-MF* to represent independently trained feature driven model, MLE-based and MLE-free point process models, respectively. Our MLE-based PreNets and MLE-free PreNets are respectively short as PreNets-MB and PreNets-MF.

4.2 Information Cascade on Twitter

Fig. 3 shows the learning curves for the point process model in PreNets-MB and PreNets-MF at adversarial training stage and the horizontal lines denote the MAPE for PP-MB and PP-MF. (Here we only show the MAPE result, and other metrics exhibit similar trends.) The curves show that as the training epoch increases, the MAPE for point process models decreases. During the adversarial training stage, the feedback signal from the 'critic' does help the point process models to promote the precision. Besides, comparing with the two pictures, we can find that two MLE-free models (PP-MF and PreNets-MF) performs better than two MLE-based models (PP-MB and PreNets-MB) in early-stage prediction, and the order reverses when it comes to prediction with long observation. One possible reason is that the estimated intensity sequences can be a good representation of the event sequences, but MLE on early-stage insufficient data would lead to inaccuracy of estimation.

Table 1 and Table 2 list the experiment results of several baselines and PreNets in prediction with short and long observation, respectively. Based on the results, we can summarize three-fold findings. i) MLE-free models are better than MLE-based models whether with adversarial training or not in early-stage prediction, and the opposite is true if the observation time becomes long enough.

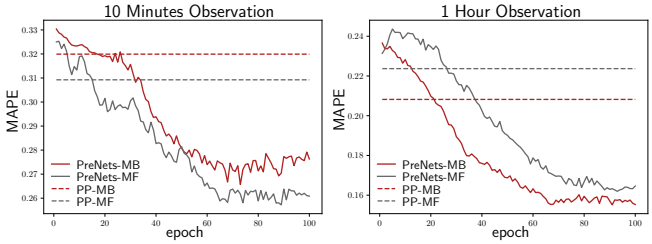

Figure 3: Learning curves for MAPE.

Table 1: 10 minutes observation for Twitter.

	MAPE	Kendall	C@50[1]	C@20
LARM	0.3088	**0.7994**	0.7807	**0.7884**
Hybrid	**0.2822**	0.7709	**0.7992**	0.7812
PP-MB	0.316	0.7586	0.6882	0.6949
PP-MF	0.2980	0.7821	0.7215	0.7556
FD	0.2862	0.7825	0.7467	0.7652
PreNets-MB	0.2811	0.7947	0.8180	0.8265
PreNets-MF	**0.2663**	**0.8147**	**0.8411**	**0.8365**

[1] The notion C@k is short for Coverage@k.

Table 2: 1 hour observation for Twitter.

	MAPE	Kendall	C@50	C@20
LARM	0.2569	0.8230	0.8659	0.8838
Hybrid	**0.1996**	0.7854	**0.9068**	0.8968
PP-MB	0.2082	**0.8270**	0.8929	**0.9029**
PP-MF	0.2237	0.8134	0.8764	0.8679
FD	0.2310	0.7922	0.8441	0.8630
PreNets-MB	**0.1574**	**0.8683**	**0.9124**	**0.9306**
PreNets-MF	0.1690	0.8321	0.8964	0.9177

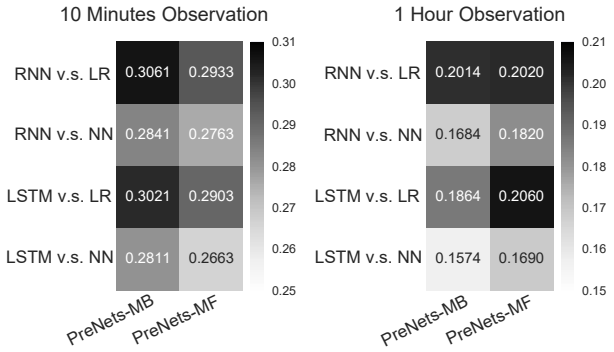

Figure 4: MAPE with different implementations for feature driven and point process models.

ii) Feature driven model performs better than point process models in early-stage prediction, while point process models achieve better precision with long observation. iii) Hybrid, PreNets-MB and PreNets-MF, as three models combining feature driven and point process perspectives achieve superior performance over other methods. For prediction with 10 minutes and 1 hour observation, our models PreNets-MB/PreNets-MF outperform all other methods, respectively achieving 0.4%/5.6% and 21.1% /15.3% improvement on MAPE.

Fig. 4 compares different model implementations for d_w, g_θ and g'_α. We compare 1-layer full-connected neural network (LR) and 3-layer ELU neural network (NN) for feature driven model d_w. Besides, we compare RNN and LSTM for point process models g_θ (MLE-based) and g'_α (MLE-free). The notion 'RNN v.s. LR' denotes that we implement point process model by LR and feature driven model by RNN. Different heat colors are used to provide a visual comparison. As we can see, more complex architectures can provide better performance. Also, changing LR to NN for feature driven model would give rise to more improvement than changing RNN to LSTM for point process models. The possible reason is that NN with more capacity could exploit more information from the features and provide better guidance to the 'interpreter'. The similar argument is given in [3].

We also apply PreNets to different observation time and study its performance variation. Fig. 5 shows the distributions of Absolute Percent Error (APE) and indicates that i) increasing observation time can reduce the median APE and also its variance, and ii) the MLE-based PreNets relies more on observation time than the MLE-free counterpart.

4.3 Movie Review on Amazon

We also run our model to predict the movie review count. At the outset, we list the basic results in Table 3 and Table 4. The experiment results are similar to those for Twitter, but have the following differences. First, the prediction performance of each method for Amazon is not as good as Twitter. We conjecture that the relatively weak correlation between the historical behaviors and the future makes it more difficult to only using temporal information to predict the future trend. Second, the LARM achieves much better precision (relative to other methods) for Amazon than Twitter. This indicates that the regression based LARM is more competent to tackle the event sequences with long life durations.

Fig. 6 shows the APE distributions for different observation time. As is depicted, the observation time plays a more important role in Amazon data set, and a small increase of the observation time would bring a big promotion of prediction precision. In comparison with Hybrid, LARM and PreNets-MB, PreNets-MF is less sensitive to observation time, which demonstrates its superiority for early-stage prediction.

Since the Amazon data set also contains the review content and user information, we extract some other features based on this information to compare the performance with only using temporal information. Event features include: the length of each review, the 'helpfulness' score of each review, the summary of each review (embedded to vectors), the history review number and averaged score of each user. Source features include: the avg. and std. of review length, the avg. and std. of 'helpfulness' scores, the sum and mean of review summary vectors, the number of influential users (whose history review number exceeds a threshold). For MLE-based point process model, event features are concatenated to the intensity sequence $\mathcal{A}^i$ as the input of g_θ. For MLE-free PreNets, we add these features to the event sequence $\mathcal{S}^i$ and input them into g'_α. For feature driven model, source features are incorporated into feature vector X^i and further input into d_w. Fig. 7 presents the experiment results, which suggest that i) other features from the perspective

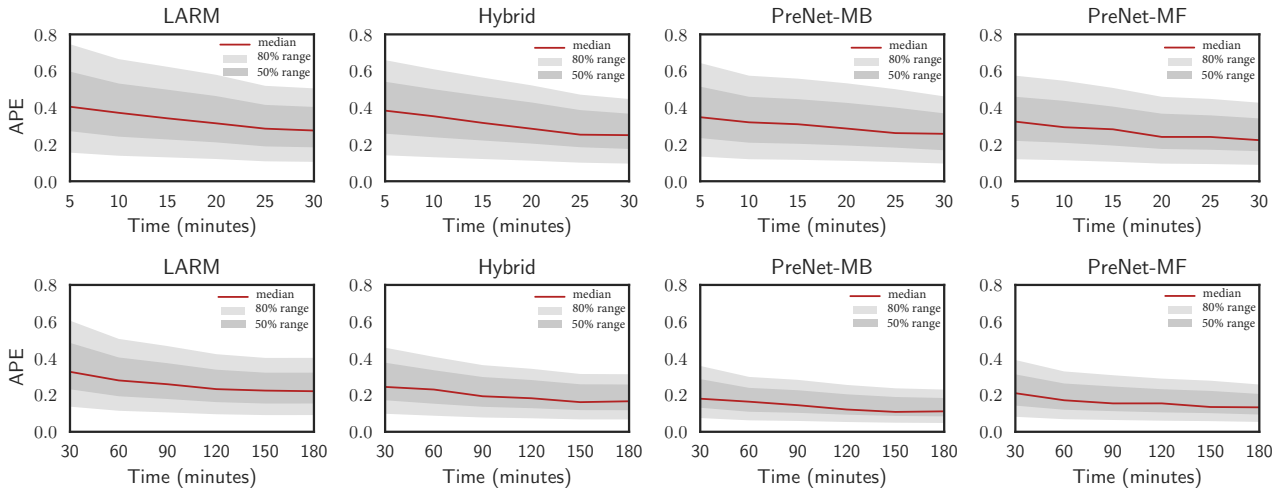

Figure 5: APE distributions for Twitter under different observation time. The red line denotes the median APE, while the light and dark gray regions respectively stand for 50th and 80th percentiles of the APE distribution.

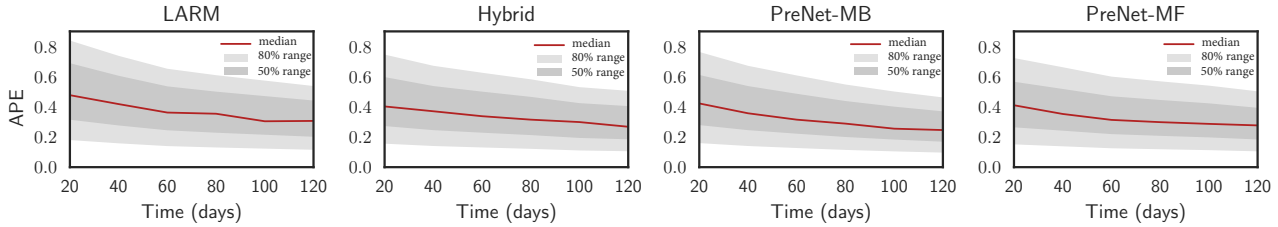

Figure 6: APE distributions for Amazon under different observation time.

Table 3: 20 days observation for Amazon.

	MAPE	Kendall	C@50	C@20
LARM	**0.3605**	**0.6786**	0.7321	0.7214
Hybrid	0.3781	0.6447	0.7341	0.7015
PP-MB	0.4244	0.6184	0.6458	0.6546
PP-MF	0.3661	0.6762	**0.7409**	**0.7285**
FD	0.4660	0.6025	0.6229	0.6591
PreNets-MB	0.3791	0.7104	0.6812	0.6790
PreNets-MF	**0.3347**	**0.7307**	**0.7577**	**0.7562**

Table 4: 100 days observation for Amazon.

	MAPE	Kendall	C@50	C@20
LARM	**0.2707**	**0.7633**	0.8356	**0.8270**
Hybrid	0.2770	0.7502	**0.8364**	0.8198
PP-MB	0.3185	0.7388	0.8236	0.8106
PP-MF	0.3323	0.6924	0.8096	0.7934
FD	0.3213	0.7209	0.8291	0.8027
PreNets-MB	**0.2536**	**0.7845**	**0.8582**	**0.8435**
PreNets-MF	0.2762	0.7719	0.8357	0.8288

of event and user information can improve the precision to a great extent, and ii) the improvement brought to early-stage prediction (about 24.3%) is much more significant than that to prediction with long observation (about 16.1%).

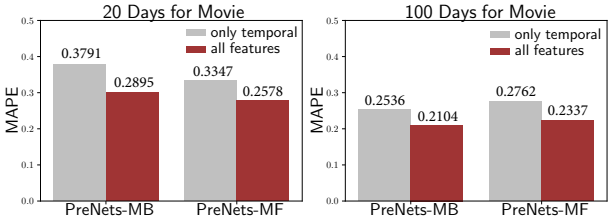

Figure 7: MAPE comparison for Amazon with only temporal features and all features.

To sum up, we summarize our empirical findings as follows. i) The adversarial training can enhance the prediction precision compared with independent training for feature driven or point process model. ii) Whether with adversarial training or independent training, the MLE-based point process model performs better than the MLE-free counterpart when the prediction is based on long observation, and the order reverses for prediction with short observation. iii) The more complex implementations for point process model and especially feature driven model can improve the precision. iv) More early features like user history records and event information can bring about precision improvement, particularly for early-stage prediction. v) Compared with the baselines, our model PreNets achieves better precision as well as less dependency on observation time and specific prediction tasks.

5 CONCLUSION

This paper proposes an adversarial training model PreNets, unifying two thinking paradigms for event popularity prediction. The framework aims at searching for an equilibrium between feature driven model, which plays as a 'critic' discriminating the predicted value from the real one, and point process model, which plays as an 'interpreter' generating a convincing predicted popularity based on the input of an observed sequence. Through a Wasserstein learning based minimax game, the precision of the 'interpreter' is enhanced with the guidance of the 'critic', leading the framework to absorb the advantages of two models. The empirical results on two different practical prediction tasks demonstrate the effectiveness and superiority of PreNets.

For future work, we will extend PreNets to other prediction tasks, such as contagion of epidemic diseases. Furthermore, as we mainly leverage temporal information to conduct prediction for generality, one possible future direction is to incorporate spatial information from social networks and delve into the spatio-temporal evolution of event sequence in the networks.

6 ACKNOWLEDGEMENTS

This work is supported by the China 973 project (2014CB340303), the National Natural Science Foundation of China (61872238, 61472252, 61672353), the Shanghai Science and Technology Fund (17510740200), the CCF-Tencent Open Research Fund (RAGR20170114), and Huawei Innovation Research Program (HO2018085286).

REFERENCES

[1] Martín Abadi and David G. Andersen. 2016. Learning to Protect Communications with Adversarial Neural Cryptography. *CoRR* abs/1610.06918 (2016).
[2] Martín Arjovsky and Léon Bottou. 2017. Towards Principled Methods for Training Generative Adversarial Networks. *CoRR* abs/1701.04862 (2017).
[3] Martín Arjovsky, Soumith Chintala, and Léon Bottou. 2017. Wasserstein Generative Adversarial Networks. In *ICML*. 214–223.
[4] Peng Bao, Huawei Shen, Xiaolong Jin, and Xueqi Cheng. 2015. Modeling and Predicting Popularity Dynamics of Microblogs using Self-Excited Hawkes Processes. In *WWW*. 9–10.
[5] Justin Cheng, Lada A. Adamic, P. Alex Dow, Jon M. Kleinberg, and Jure Leskovec. 2014. Can cascades be predicted?. In *WWW*. 925–936.
[6] Rodrigo Augusto da Silva Alves, Renato Martins Assunção, and Pedro Olmo Stancioli Vaz de Melo. 2016. Burstiness Scale: A Parsimonious Model for Characterizing Random Series of Events. In *SIGKDD*. 1405–1414.
[7] D.J. Daley and David Vere-Jones. 2007. An Introduction to the Theory of Point Processes. *Springer* (2007).
[8] Nan Du, Hanjun Dai, Rakshit Trivedi, Utkarsh Upadhyay, Manuel Gomez-Rodriguez, and Le Song. 2016. Recurrent Marked Temporal Point Processes: Embedding Event History to Vector. In *SIGKDD*. 1555–1564.
[9] Mehrdad Farajtabar, Yichen Wang, Manuel Gomez-Rodriguez, Shuang Li, Hongyuan Zha, and Le Song. 2015. COEVOLVE: A Joint Point Process Model for Information Diffusion and Network Co-evolution. In *NIPS*. 1954–1962.
[10] Shuai Gao, Jun Ma, and Zhumin Chen. 2014. Effective and effortless features for popularity prediction in microblogging network. In *WWW*. 269–270.
[11] Manuel Gomez-Rodriguez, Jure Leskovec, and Bernhard Schölkopf. 2013. Modeling Information Propagation with Survival Theory. In *ICML*. 666–674.
[12] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron C. Courville, and Yoshua Bengio. 2014. Generative Adversarial Nets. In *NIPS*. 2672–2680.
[13] Alex Graves, Marcus Liwicki, S. Fernandez, Roman Bertolami, Horst Bunke, and Jürgen Schmidhuber. 2009. A Novel Connectionist System for Unconstrained Handwriting Recognition. *IEEE Trans. Pattern Anal. Mach. Intell.* 31, 5 (2009), 855–868.
[14] Ishaan Gulrajani, Faruk Ahmed, Martín Arjovsky, Vincent Dumoulin, and Aaron C. Courville. 2017. Improved Training of Wasserstein GANs. In *NIPS*. 5769–5779.
[15] Hideaki Kim, Noriko Takaya, and Hiroshi Sawada. 2014. Tracking Temporal Dynamics of Purchase Decisions via Hierarchical Time-Rescaling Model. In *CIKM*. 1389–1398.
[16] Shoubin Kong, Qiaozhu Mei, Ling Feng, Fei Ye, and Zhe Zhao. 2014. Predicting bursts and popularity of hashtags in real-time. In *SIGIR*. 927–930.
[17] Jiwei Li, Will Monroe, Tianlin Shi, Sébastien Jean, Alan Ritter, and Dan Jurafsky. 2017. Adversarial Learning for Neural Dialogue Generation. In *EMNLP*. 2157–2169.
[18] Sha Li, Xiaofeng Gao, Weiming Bao, and Guihai Chen. 2017. FM-Hawkes: A Hawkes Process Based Approach for Modeling Online Activity Correlations. In *CIKM*. 1119–1128.
[19] Xiao Lin, Min Zhang, Yongfeng Zhang, Yiqun Liu, and Shaoping Ma. 2017. Learning and Transferring Social and Item Visibilities for Personalized Recommendation. In *CIKM*. 337–346.
[20] Changsha Ma, Zhisheng Yan, and Chang Wen Chen. 2017. LARM: A Lifetime Aware Regression Model for Predicting YouTube Video Popularity. In *CIKM*. 467–476.
[21] Xiao Ma, Xiaofeng Gao, and Guihai Chen. 2017. BEEP: A Bayesian Perspective Early Stage Event Prediction Model for Online Social Networks. In *ICDM*. 973–978.
[22] Swapnil Mishra, Marian-Andrei Rizoiu, and Lexing Xie. 2016. Feature Driven and Point Process Approaches for Popularity Prediction. In *CIKM*. 1069–1078.
[23] Yosihiko Ogata. 1981. On Lewis' simulation method for point processes. *IEEE Trans. Information Theory* 27, 1 (1981), 23–30.
[24] Sasa Petrovic, Miles Osborne, and Victor Lavrenko. 2011. RT to Win! Predicting Message Propagation in Twitter. In *ICWSM*.
[25] Scott E. Reed, Zeynep Akata, Xinchen Yan, Lajanugen Logeswaran, Bernt Schiele, and Honglak Lee. 2016. Generative Adversarial Text to Image Synthesis. In *ICML*. 1060–1069.
[26] Marian-Andrei Rizoiu, Young Lee, Swapnil Mishra, and Lexing Xie. 2017. A Tutorial on Hawkes Processes for Events in Social Media. *CoRR* abs/1708.06401 (2017).
[27] Marian-Andrei Rizoiu, Swapnil Mishra, Quyu Kong, Mark James Carman, and Lexing Xie. 2018. SIR-Hawkes: Linking Epidemic Models and Hawkes Processes to Model Diffusions in Finite Populations. In *WWW*. 419–428.
[28] Tim Salimans, Ian J. Goodfellow, Wojciech Zaremba, Vicki Cheung, Alec Radford, and Xi Chen. 2016. Improved Techniques for Training GANs. In *NIPS*. 2226–2234.
[29] Bidisha Samanta, Abir De, Abhijnan Chakraborty, and Niloy Ganguly. 2017. LMPP: A Large Margin Point Process Combining Reinforcement and Competition for Modeling Hashtag Popularity. In *IJCAI*. 2679–2685.
[30] Hua-Wei Shen, Dashun Wang, Chaoming Song, and Albert-László Barabási. 2014. Modeling and Predicting Popularity Dynamics via Reinforced Poisson Processes. In *AAAI*. 291–297.
[31] Benjamin Shulman, Amit Sharma, and Dan Cosley. 2016. Predictability of Popularity: Gaps between Prediction and Understanding. In *ICWSM*. 348–357.
[32] Bongwon Suh, Lichan Hong, Peter Pirolli, and Ed H. Chi. 2010. Want to be Retweeted? Large Scale Analytics on Factors Impacting Retweet in Twitter Network. In *IEEE SocialCom*. 177–184.
[33] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to Sequence Learning with Neural Networks. In *NIPS*. 3104–3112.
[34] David Vallet, Shlomo Berkovsky, Sebastien Ardon, Anirban Mahanti, and Mohamed Ali Kâafar. 2015. Characterizing and Predicting Viral-and-Popular Video Content. In *CIKM*. 1591–1600.
[35] Duy Quang Vu, Arthur U. Asuncion, David R. Hunter, and Padhraic Smyth. 2011. Dynamic Egocentric Models for Citation Networks. In *ICML*. 857–864.
[36] Jun Wang, Lantao Yu, Weinan Zhang, Yu Gong, Yinghui Xu, Benyou Wang, Peng Zhang, and Dell Zhang. 2017. IRGAN: A Minimax Game for Unifying Generative and Discriminative Information Retrieval Models. In *SIGIR*. 515–524.
[37] Pengfei Wang, Yanjie Fu, Guannan Liu, Wenqing Hu, and Charu C. Aggarwal. 2017. Human Mobility Synchronization and Trip Purpose Detection with Mixture of Hawkes Processes. In *KDD*. 495–503.
[38] Qun Wu, Tian Wang, Yiqiao Cai, Hui Tian, and Yonghong Chen. 2017. Rumor restraining based on propagation prediction with limited observations in large-scale social networks. In *ACSW*. 1:1–1:8.
[39] Shuai Xiao, Mehrdad Farajtabar, Xiaojing Ye, Junchi Yan, Xiaokang Yang, Le Song, and Hongyuan Zha. 2017. Wasserstein Learning of Deep Generative Point Process Models. In *NIPS*. 3250–3259.
[40] Shuai Xiao, Junchi Yan, Xiaokang Yang, Hongyuan Zha, and Stephen M. Chu. 2017. Modeling the Intensity Function of Point Process Via Recurrent Neural Networks. In *AAAI*. 1597–1603.
[41] Jiang Yang and Scott Counts. 2010. Predicting the Speed, Scale, and Range of Information Diffusion in Twitter. In *ICWSM*.
[42] Linyun Yu, Peng Cui, Fei Wang, Chaoming Song, and Shiqiang Yang. 2015. From Micro to Macro: Uncovering and Predicting Information Cascading Process with Behavioral Dynamics. In *ICDM*. 559–568.
[43] Qingyuan Zhao, Murat A. Erdogdu, Hera Y. He, Anand Rajaraman, and Jure Leskovec. 2015. SEISMIC: A Self-Exciting Point Process Model for Predicting Tweet Popularity. In *SIGKDD*. 1513–1522.